

Team teaching material

Bastien Chassagnol, Benjamin Loire, Morgane, Philippe, Marielle

Table of contents

1. Welcome	1
2. Team onboarding	3
2.1. Logistics	3
2.1.1. Communication channels	3
2.1.2. Calendars	3
2.1.3. Mandatory meetings	4
2.1.4. Working documents and sharing	5
2.1.5. Good habits for reports and notes	5
2.1.6. Mailing lists	6
2.1.7. Additional information	6
2.2. Outreach initiatives	7
I. Fresh Ubuntu installation	9
3. Ubuntu general guidelines	11
3.1. Preamble: recommended system libraries to install	11
3.2. Ubuntu file and package structure	11
3.2.1. System-wide location (for core programs)	11
3.2.2. User-specific locations	12
3.3. Recommended updating policy of Ubuntu	18
3.4. Monitor processes running in the background, and on startup	19
3.5. Network protocols, and connection troubleshoots	20
3.5.1. Symptom: Wi-Fi connects, but some websites fail . .	20
3.5.2. Permanent fix	22

Table of contents

3.6. Pimp your shell: switch from boring Bash to fancy Zsh . . .	22
4. Software to install	27
4.1. Git and GitHub configuration	30
4.2. PDF management	31
4.2.1. Sioyek as PDF viewer	31
4.2.2. Sejda for interactive PDF editing	34
4.2.3. PDFtk	34
4.3. Image editors (to replace Paint on Windows)	35
4.4. LibreOffice	35
4.5. Docker	40
4.6. Web browsers	41
4.6.1. Firefox	41
4.6.2. Google Chrome	42
4.7. Slack	43
4.8. Zulip	43
4.9. Cytoscape	44
4.10. Zotero	45
4.11. Quarto, R and Cursor	48
4.11.1. R and recommended packages	48
4.11.2. Quarto	50
4.11.3. Cursor with R	50
4.12. Python	50
4.12.1. Modern Python guidelines	52
4.12.2. One tool to bind them: a uv workflow	55
4.12.3. Marimo (preferred over Jupyter Lab)	57
4.13. Nextflow	57
4.14. LaTeX	57
4.15. Cursor	57
4.16. Password manager: Bitwarden	59
5. AMU tools and remote servers	65
5.1. Core numerical tools provided by AMU	65
5.1.1. AMU VPN (GlobalProtect)	65

5.1.2. AMUZoom	66
5.1.3. WooClap for QCMs	68
5.2. Working with a remote server	68
5.2.1. SSH configuration	69
5.2.2. MMG cluster and recommendations for project management	72
Appendices	75
A. OneDrive on Linux	75
A.1. Open-source installation for Ubuntu	75
A.2. Permanent monitoring: pick exactly one path	76
A.2.1. Option A — systemd + journalctl	76
A.2.2. Option B — tray GUI	78
A.3. Troubleshoot OneDrive	79
A.3.1. Safe recovery after a messy sync	80
B. R programming guidelines	87
B.1. Package Structure	87
B.2. Project Setup (End-to-End)	88
B.3. Writing Functions	88
B.3.1. Input checking	89
B.4. Dependencies and Namespaces	89
B.5. Documentation with roxygen2	90
B.6. Testing with testthat	91
B.7. Data in Packages	92
B.8. Build, Check, and Release	92
B.9. Documenting Multiple Functions on One Help Page	93
B.10. Parallel Computing in R (Practical Notes)	94
B.11. Lifecycle R development	94
B.11.1. 1) High-level package lifecycle	95
B.11.2. 2) What changes between source, bundle, and binary	97
B.11.3. 3) Command-driven workflow (construction to load)	99

Table of contents

C. Modern LaTeX guidelines	101
C.1. Document structure	101
C.2. Spacing and paragraphs	102
C.3. Mathematical typesetting	102
C.4. Text formatting	103
C.5. Figures and tables	103
C.6. Bibliography and citations	103
C.7. Cross-references	104
C.8. Lists	104
C.9. Recommended packages	104
C.10. Quarto LaTeX mapping	105
C.11. Compilation and quality	106
C.12. Anti-patterns	106

1. Welcome

This site is built with Quarto as a **book** project. Use the table of contents to browse:

- Team onboarding
- Ubuntu general guidelines
- Software to install
- AMU tools and remote servers
- Appendices: OneDrive, R package guidelines, Modern LaTeX

Download the compiled book as a single PDF: [team-teaching-material.pdf](#).

For other materials in this repository (SNF, Git, package development, etc.), see the project [README.md](#) in the repository root.

2. Team onboarding

Practical logistics for joining the lab (Marielle Péré’s group and the broader MMG / MABIOS ecosystem). Prefer British English in written notes and reports. For programming and Linux setup, see the other chapters of this book; for a longer living document, see also the shared onboarding Google Doc and the Onboarding Canva on Slack.

2.1. Logistics

2.1.1. Communication channels

- **Preferred channel: Slack.** On your first day you will be added to the team workspace. Configure notification schedules (for example mute outside 09:00–17:00) so you stay reachable without burning out — **do this early.**
- Urgent / personal coordination with Marielle: +33 7 86 45 45 75.
- Routine meetings are listed below. For any **extra** meeting, ask first, then add it to the calendar.

2.1.2. Calendars

- **MP personnel calendar:** you will be added to Marielle’s shared Google Calendar. Meetings are often labelled with initials (for example MP for Marielle Péré). Add your own meetings there as well.
- **Team calendar:** Google Calendar (team)

2. Team onboarding

- **MMG events:** Google Calendar (MMG)

2.1.3. Mandatory meetings

Meeting	When	Where / notes
DTOO	Monday 13:30 (~1.5 h)	MMG or Luminy (same site as the team meeting that day). Bring slides summarising what you did and what you plan next.
Team meeting (MABIOS + Paul Villoutreix's team)	Monday 15:00	MMG, or Luminy about once a month. Topic announced ~1 week ahead by Solène (Paul's team): talk, journal club, or logistics.
MMG internal seminar	Thursday 13:00	MMG. A unit member (PhD, post-doc, or permanent) presents their work.
MMG guest seminar	Usually Tuesday morning (check calendar)	External speaker; announced several weeks ahead.
Daily stand-up	~5 minutes each morning	With Marielle: yesterday / today.
Weekly 1:1	Longer slot once per week	Scheduled on day one.

Times are Europe/Paris unless stated otherwise.

2.1.4. Working documents and sharing

Need	Tool
Day-to-day links, papers, quick notes	Slack
Lab-meeting slides and shared archives	Resana (login)
Collaborative writing (LaTeX)	Overleaf (Anaïs has a professional account — ask her to create/share a project)
Lightweight prose	Google Docs
Presentations	Overleaf if maths-heavy; otherwise PowerPoint or Canva . Upload final decks to Resana.
Graphics	Canva, Servier Medical Art, Inkscape ; BioRender only if the lab covers the licence
Code	GitHub — personal repos for visibility, plus the team organisation (ask Morgane if you are not added). Maintain a serious <code>README.md</code> (see also Section 2.1.5 and the R package chapter).

2.1.5. Good habits for reports and notes

1. Spend ~**15 minutes at the end of each day** writing what you did, why, what you obtained, and what comes next.
2. On day one, a **Canva** board is created with Anaïs and Marielle on Slack. Each week (usually Friday), add a short **5:15-style report** there, in addition to the DTOO slides.
3. Keep package / project `README` files current: versions, install steps, architecture, and docstrings / roxygen.
4. A second Canva board stores **meeting notes** — update it after each meeting.

2. Team onboarding

5. Keep your bibliography in **Zotero** (see the software chapter for Linux setup).

2.1.6. Mailing lists

If you lean toward computational / mathematical biology, subscribe to relevant communities (French GdR lists and international societies), for example:

- MathOnco Weekly (mathematical modelling in biology)
- ESMTB (European Society for Mathematical and Theoretical Biology)
- SMB (Society for Mathematical Biology)
- MATH-BIO-SANTÉ (CNRS)
- JOBIM announcements
- SFBI bioinfo list: https://www.sfbi.fr/pages/listes_diffusion/
- Examples of list addresses used in the lab network: `gdr-ifm-gt-bioss-request@listes.lip6.fr`, `merit-request@services.cnrs.fr`, `gt-bioss@gdr-ifm.fr`, `bioinfo@sfbi.fr`, `math-bio-sante@listes.math.cnrs.fr`, `bull-ia@gdria.fr`

2.1.7. Additional information

Door codes, admin contacts, GitHub org details, and writing tips also live on Slack (Onboarding Canva) and in the shared Google Doc.

! Important

Set Slack notification preferences on day one so evenings and weekends stay quiet unless you choose otherwise.

2.2. Outreach initiatives

Contact **Marie-Cécile Gaillard** at MMG to join the outreach group. MMG is an active place for public engagement. Examples of programmes:

Initiative	Notes / links
Sciences dans les classes (Fête de la Science) / Les Petits Débrouillards	School visits; establishments usually register from June. Académie Aix-Marseille. Contacts: Aurélie Privé Hanouille, Lucille Rombi, Benjamin Vierende.
1000 chercheurs dans les écoles (AFM Téléthon)	https://www.afm-telethon.fr/fr/1000-chercheurs-dans-les-ecoles
La Science taille XX elles (France / Marseille)	National; Marseille CNRS
Femmes & Sciences	https://www.femmesetsciences.fr/femmes-inspirantes
MarMaRa / Festival des Arts et des Sciences	Institute participation (e.g. 2026 call): festival call
Pint of Science	Public talks in pubs / cafés
Chiche! 1 classe, 1 chercheur	Classroom researcher visits
Terra Numerica	Stronger presence around Sophia Antipolis
Fête de la Science	National science festival
Recherch'thon	Awareness / fundraising research events
Les Petits Débrouillards	Hands-on science for young audiences
Les Éclaireuses	Mentoring / visibility initiatives
Math en Jean	Maths workshops with secondary schools
Maths & Filles — Une équation lumineuse	Encouraging girls in mathematics

2. Team onboarding

Ask Marie-Cécile or Marielle which actions are active this semester before committing calendar time.

Part I.

Fresh Ubuntu installation

3. Ubuntu general guidelines

- `dpkg --print-architecture`: retrieve architecture, for package installation¹.
- `Ctrl-H` in **G**nome, aka File Management, to list hidden files

3.1. Preamble: recommended system libraries to install

3.2. Ubuntu file and package structure

3.2.1. System-wide location (for core programs)

System-wide locations affect all users. Software is installed with `sudo`, on the root of the system.

Location	Purpose
<code>/usr/bin</code>	System executables (APT)
<code>/etc</code>	System-wide configuration (example: define repositories for <code>apt</code>)
<code>/opt</code>	Vendor (external) apps
<code>/usr/local/bin</code>	Admin-installed software

¹Returns `amd64` in my case

3. Ubuntu general guidelines

Location	Purpose
<code>/var</code>	Logs, caches, databases

Table 3.1.: Main system-wide software locations

- `/opt` for third-party application bundles, **self-contained**, while `/usr/local` is for third-party **binaries**. Besides, `/opt` is not in the path by default.

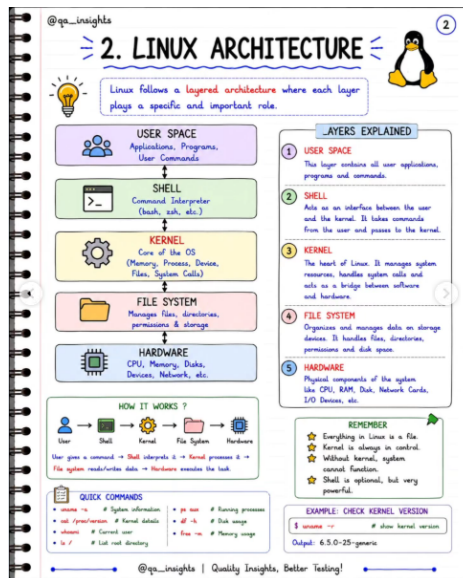
3.2.2. User-specific locations

The user does not require root.

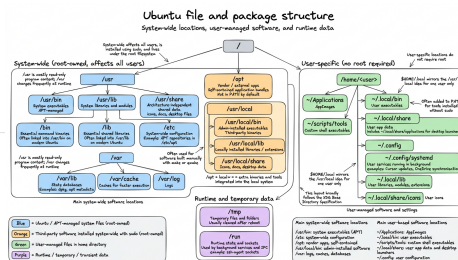
Location	Purpose
<code>~/Applications</code>	AppImages
<code>~/.local/bin</code>	User executables
<code>~/scripts/tools</code>	Custom bash executables
<code>~/.local/share</code>	User app data, notably <code>~/.local/share/applications</code> storing desktop icons
<code>~/.config</code>	User configuration

Table 3.2.: Main user-based software locations, to be compared with Table 3.1

3.2. Ubuntu file and package structure



- (a) How Linux processes shell commands:
User → Shell → Kernel → File system
→ Hardware



- (b) Directory roles as a “city” map (every path has a job)

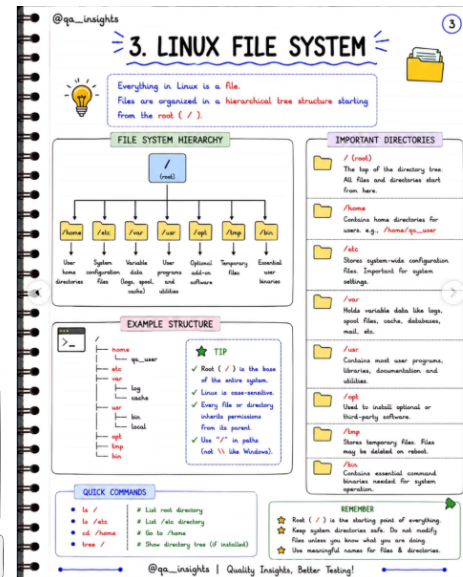


Figure 3.1.: Linux at a glance. **Left:** how a shell command travels through OS layers (user space → shell → kernel → file system → hardware). **Right:** filesystem layout — Ubuntu package paths, the hierarchical tree from /, and directory roles as a city map. Illustrations inspired by @qa_insights (Quality Insights) on Instagram; directory “city” framing in the same teaching-poster style. For fundamentals, see also the Filesystem Hierarchy Standard (FHS), the XDG Base Directory Specification, and the Linux Documentation Project.

3. *Ubuntu general guidelines*

i Main folders and path roles

Everything under Linux hangs from the **root** / (a forward slash — paths are case-sensitive). Think of the tree as a city map: each top-level directory has a fixed job (Figure 3.1).

Core hierarchy

3.2. Ubuntu file and package structure

Path	Role
/	Root of the tree; all absolute paths start here.
/home	Per-user homes (e.g. /home/<user>); your documents and configs live here.
/root	Home directory of the root administrator (not the same as /).
/etc	System-wide configuration (apt sources, network, services).
/bin, /sbin	Essential command binaries (user vs admin-oriented). On modern Ubuntu many of these are merged into /usr.
/usr	Read-only applications, libraries, and shared data installed by the distribution.
/opt	Optional / third-party self-contained application bundles (often not on \$PATH until you add a symlink).
/var	Variable data that grows over time: logs, caches, databases, spools.
/tmp	Temporary files; often cleared on reboot.
/mnt, /media	Mount points for disks, USB drives, and other external storage.
/dev	Device nodes (hardware exposed as files).
/proc, /sys	Live kernel / process / hardware interfaces (virtual filesystems).
/lib, /lib64	Shared libraries needed by core programs.
/boot	Kernel and bootloader files.
/run	Runtime state (PID files, sockets such as ssh-agent).
/lost+found	Recovered fragments after filesystem checks.

3. Ubuntu general guidelines

/usr versus /var

- /usr is meant for *mostly read-only* executables and shared assets shipped by packages.
- /var is written continuously:
 - /var/lib: state databases (e.g. dpkg, apt)
 - /var/cache: downloadable / rebuildable caches
 - /var/log: service and system logs (first stop when an app misbehaves)

Custom builds and per-user tools

- /usr/local/ (system-wide, needs sudo) and \$HOME/.local (user-only) hold software built with make / qmake or installed outside APT:
 - bin, sbin — executables
 - share — icons (icons/), desktop entries, docs (doc/)
 - lib — extensions (R packages, Python modules, shared objects)
 - User layout follows the XDG Base Directory Specification (~/.config, ~/.local/share, ~/.cache, ...)

Runtime and background services

- /tmp — scratch space; do not store lasting work there.
- /run — sockets and ephemeral service state (e.g. ssh-agent).
- ~/.config/systemd — user units that keep running after login (Cursor updates, OneDrive sync, ...).

Memory aid

Start at / → configure in /etc → run tools from /bin or /usr → optional apps in /opt → inspect logs in /var → monitor live state via /proc and /sys.

3.2. Ubuntu file and package structure

Listing 3.1 Install recommended system libraries

```
# Install system libs
sudo apt-get update
sudo apt-get install -y \
    apt-transport-https \
    build-essential \
    ca-certificates \
    cloud-guest-utils \
    curl \
    default-jre \
    f2c \
    gdebi-core \
    gnupg \
    libbz2-dev \
    libcurl4-openssl-dev \
    libffi-dev \
    libfontconfig1-dev \
    libfribidi-dev \
    libgsl0-dev \
    libgtextutils-dev \
    libharfbuzz-dev \
    libicu-dev \
    liblzma-dev \
    libncurses5-dev \
    libncursesw5-dev \
    libpcre3 \
    libpcre3-dev \
    libperl-dev \
    libreadline-dev \
    libssl-dev \
    libtiff-dev \
    libxml2-dev \
    locales \
    ncftp \
    pandoc \
    parallel \
    pkg-config \
    readline-doc \
    sudo \
    tcl \
    tk \
    tk-dev \
    tk-table \
    tmux \
    tzdata \
    unzip \
    vim \
    && sleep 1
```

3. Ubuntu general guidelines

3.3. Recommended updating policy of Ubuntu

1. Use the maintained helper script `ubuntu/update_os.sh`. It runs the usual `apt update / upgrade / cleanup` path, but **refuses kernel package changes when any installed DKMS module is missing or broken** for the running (or newest) kernel — so a partial upgrade cannot leave you without drivers such as webcam relays, NVIDIA, or VirtualBox.
2. Make it executable and run it with admin rights:

Listing 3.2 Run the Ubuntu update helper

```
chmod +x ubuntu/update_os.sh
sudo ./ubuntu/update_os.sh
```

To force a kernel upgrade anyway (only if you accept a broken DKMS stack): `sudo FORCE_KERNEL_UPGRADE=1 ./ubuntu/update_os.sh`.

3. Reboot the system with `reboot` when a new kernel was installed.



Ubuntu partial updates

Partial updates breaking key driver components occur more often with Ubuntu compared to Microsoft, since Linux tends to provide lots of separate, modular packages while Windows ships large and standalone bundles — **Consequence: more issues with updates, especially if the network connection dropped, and the newest versions of the packages were not fully installed.**

General recommendations:

3.4. Monitor processes running in the background, and on startup

```
# to be run daily
sudo apt update
sudo apt upgrade

# to be run monthly
sudo apt full-upgrade

## clean deprecated dependencies
sudo apt autoremove
sudo apt autoclean
```

Deal with driver issues

```
## Check for Broken Packages
sudo dpkg --configure -a
## Install missing dependencies
sudo apt -f install

## List all Hardware Driver Errors
dmesg -l err,crit,alert,emerg
```

3.4. Monitor processes running in the background, and on startup

- `systemctl list-unit-files --type=service --state=running --user` to list all processes running in the background, even after rebooting, restrained to the user-level.
 - Alternatively, processes at the system-level are stored under

3. Ubuntu general guidelines

`/lib/systemd/system/`

- `systemctl status <service>` retrieves current status

3.5. Network protocols, and connection troubleshoots

- Two IP protocols for Wi-Fi: IPv4 versus IPv6.



The two Internet protocols, namely IPv4 vs IPv6

IPv4 is older, but still prevailing in many websites, such as GitHub, or OneDrive:

- Uses 32-bit addresses (e.g. `192.168.1.10`)
- Used by **most websites**, CDNs, VPNs, corporate services
- But the 32-bit limitation hinders the number of available IP addresses.

IPv6 is newer, but not universally supported:

- Uses 128-bit addresses (e.g. `2a01:cb14:86a7::1`)
- Supported by modern ISPs and operating systems
- Some networks provide **IPv6-only**, with no IPv4 fallback. Especially the case with Microsoft services, governmental or academic websites.

3.5.1. Symptom: Wi-Fi connects, but some websites fail

On Ubuntu **24.04 (Noble)**, NetworkManager may show the link as connected while `ip` lists **only IPv6** addresses. Sites that need IPv4 (or that resolve poorly without it) then fail — e.g. `curl https://github.com` or `curl https://onedrive.live.com` hang or return *Cannot connect*. Eduroam or

3.5. Network protocols, and connection troubleshoots

a phone hotspot often work; some home routers (e.g. Orange Livebox) expose the bug clearly.

Root cause (not “IPv6-only router”): under Noble, NetworkManager’s DHCP helper lives at `/usr/libexec/nm-dhcp-helper`, but the stock AppArmor profile for `dhclient` still allows only the old path `/usr/lib/NetworkManager/nm-dhcp-helper`. `dhclient` obtains a lease (DHCP OFFER / DHCPACK), then AppArmor **denies** the helper → NetworkManager never applies IPv4²

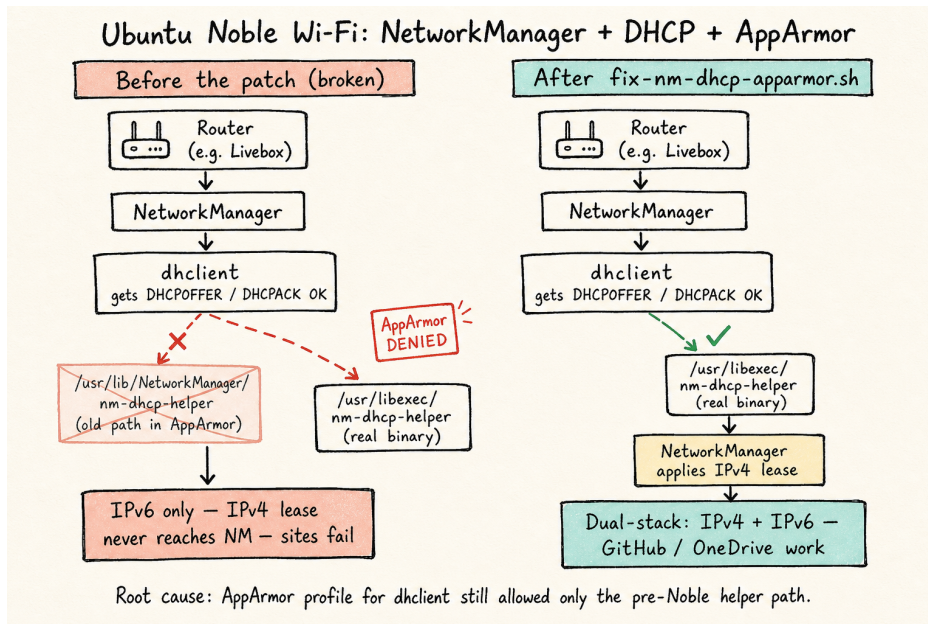


Figure 3.2.: Wi-Fi DHCP under Ubuntu Noble: AppArmor blocks the moved `nm-dhcp-helper` until the profile is patched

²See Figure 3.2 and Launchpad bugs #2083017, #2116733.

3. Ubuntu general guidelines

3.5.2. Permanent fix

1. Run Listing 3.3. The script in Listing 3.4 (file: `ubuntu/fix-nm-dhcp-apparmor.sh`) patches `/etc/apparmor.d/sbin.dhclient` for `/usr/libexec/nm-dhcp-helper`, sets NetworkManager to `dhcp=dhclient`, softens Wi-Fi powersave, reloads AppArmor, and restarts NetworkManager.

Listing 3.3 Make the AppArmor DHCP fix executable and run it

```
chmod +x ubuntu/fix-nm-dhcp-apparmor.sh
sudo ./ubuntu/fix-nm-dhcp-apparmor.sh
```

Listing 3.4 `fix-nm-dhcp-apparmor.sh` — allow `/usr/libexec/nm-dhcp-helper` under AppArmor

1. Reconnect to the Wi-Fi network using the GUI, then check the success of the patch:

```
ip -br addr show <wifi-network-name> # e.g. wlp2s0f0
ping -4 -c 2 1.1.1.1 # should return "64 bytes from 1.1.1.1: icmp_seq=1 ttl=55 time=1
curl -4 -sI --max-time 8 https://github.com | head -3 # should return "HTTP/2 200"
```

3.6. Pimp your shell: switch from boring Bash to fancy Zsh

1. Installation script:

3.6. Pimp your shell: switch from boring Bash to fancy Zsh

Listing 3.5 Install Zsh, set it as default shell, and install Oh My Zsh

```
sudo apt update
sudo apt install zsh

## change the default shell shebang
chsh -s $(which zsh) ## modify /etc/passwd repository
reboot # required log out for effective changes

## check if zsh is used by default (also the theme is modified)
getent passwd "$USER" # returns "/home/bastien:/usr/bin/zsh"

## Install advanced customisation of the shell with Oh My Zsh addon
sh -c "$(curl -fsSL https://raw.githubusercontent.com/ohmyzsh/ohmyzsh/master/tools/install.sh)" # cre
```

You should keep the original `~/.bashrc` as a fallback for Bash.

2. Install the modern alternative to `ls`, namely `eza`
3. Change the behaviour of the shell:
 - **System-wide:** all executables installed under `/usr` folder can be called directly from the `$PATH`. For third-party apps stored under `/opt`, you must **add a symlink** redirecting to your local folder: `sudo ln -s /opt/<appName> /usr/local/bin/<appName>`.
 - **User-level:** customise `.bashrc` (and `.zshrc` if using the `.zsh` fancy shebang) — **Only affects the current user**, see my own `.zshrc` configuration file, and below `ls` output Figure 3.3.

Useful resources:

- shell shortcuts

3. Ubuntu general guidelines

```
➤ teaching_material git:(main) X ls
Config  Git      PackageDevelopment  SNF      d_quarto.yml  README.md
docs    Linux    'Server Management' TCGA_data  index.qmd     teaching_material.Rproj
```

(a) Modified behaviour of the `ls` output

```
➔ teaching_material git:(main) x ll
Permissions User Date Modified Git Name
drwxr-xr-x - bastien 23 Apr 14:17 - [C]Config
drwxr-xr-x - bastien 28 Mar 18:34 - [C]docs
drwxr-xr-x - bastien 29 Mar 02:11 - [C]git
drwxr-xr-x - bastien 23 Apr 14:17 - [C]linux
drwxr-xr-x - bastien 21 Mar 19:39 - [C]PackageDevelopment
drwxr-xr-x - bastien 21 Mar 19:39 - [C]'Server Management'
drwxr-xr-x - bastien 21 Mar 19:39 - [C]SNF
drwxr-xr-x - bastien 31 Mar 14:54 - [C]TCGA_data
-rw-r--r-- 34k bastien 28 Mar 18:34 - d_quarto.yml
-rw-r--r-- 40k bastien 27 Mar 19:39 - d_quarto.xml
-rw-r--r-- 1.2k bastien 21 Mar 19:39 - [C]README.md
-rw-r--r-- 205 bastien 23 Apr 13:33 - [C]teaching_material.Proj
```

- (b) Modified behaviour of the `ll` output, displaying in addition to file names their sizes, permission rights, and original owner

```

# teaching_material git(main) X la
Permissions Size User Date Modified Git Name
drwxr-xr-x - bastien 21 Mar 19:39 -- I git
drwxr-xr-x - bastien 21 Mar 19:39 -- I .github
drwxr-xr-x - bastien 23 Apr 13:33 -- I .quarto
drwxr-xr-x - bastien 5 Jan 11:38 -- I .Rproj.user
drwxr-xr-x - bastien 23 Apr 14:17 -- I Config
drwxr-xr-x - bastien 28 Mar 10:34 -- I docs
drwxr-xr-x - bastien 29 Mar 02:11 -- I Git
drwxr-xr-x - bastien 23 Apr 14:18 -- M Linux
drwxr-xr-x - bastien 21 Mar 19:39 -- I PackageDevelopment
drwxr-xr-x - bastien 21 Mar 19:39 -- I 'Server management'
drwxr-xr-x - bastien 21 Mar 19:39 -- I TNCF
drwxr-xr-x - bastien 31 Mar 14:54 -- I TGA_data
-rw-rw-r-- 167 bastien 21 Mar 19:39 -- I .gitignore
-rw-rw-r-- 8 bastien 21 Mar 19:39 -- I .nojekyll
-rw-rw-r-- 1.6K bastien 10 Apr 11:02 -- I .Rhistory
-rw-rw-r-- 3.4K bastien 20 Mar 18:34 -- I _quarto.yml
-rw-rw-r-- 408 bastien 27 Mar 19:32 -- I Index.qnd
-rw-rw-r-- 1.2K bastien 21 Mar 19:39 -- I README.md
-rw-rw-r-- 205 bastien 23 Apr 13:33 -- I teaching_material.Rproj

```

- (c) The `la` output displays in addition to the `ll` output the hidden files (traditionally starting with a `.`)

```
teaching_material.git:(main) $ ls
├── .
├── .git
├── .quarto.yml
├── Config
│   ├── better_bibtex_preferences.json
│   ├── linux_cursor
│   ├── ln2ndrive-config
│   ├── RStudio
│   ├── ssh-hosts
│   └── zsh
├── docs
│   ├── Config
│   ├── Git
│   ├── Index.html
│   ├── Linux
│   ├── robots.txt
│   ├── search.json
│   ├── 'Server Management'
│   ├── site_libs
│   ├── sitemap.xml
│   └── team-teaching-material.pdf
├── Git
│   ├── diapo_formation_git.pdf
│   ├── formation_git_2024.pdf
│   ├── formation_git_solutions.pdf
│   ├── git_user_guide.pdf
│   ├── 'git basic commands.pdf'
│   ├── GitKraken-Git-Cheat-Sheet-082025.pdf
│   ├── README.md
│   └── slides_formation_git.pdf
├── index.qmd
└── Linux
    ├── extensions
    │   ├── bash
    │   ├── git_diffs_encodings.png
    │   ├── installation-linux.png
    │   ├── installation-linux.qmd
    │   ├── poetry_vs_uv_vs_pipx.pdf
    │   ├── python-dependencies-use-cases.png
    │   ├── python-development-layers.png
    │   ├── python-dust-bin.png
    │   ├── python-managers-summary-table.png
    │   ├── R_options_1f.png
    │   ├── Rstudio-config-1.png
    │   ├── Rstudio-config-2.png
    │   ├── ssh-initiate-connection.png
    │   ├── typographic_options.png
    │   ├── ubuntu_guidelines_installation.png
    │   ├── ubuntu_package_structure.png
    │   ├── 'uv cheatsheet.png'
    │   ├── zoom_options_enu_1.png
    │   ├── zoom_options_enu_2.png
    │   ├── zotero-advanced-configuration.png
    │   └── Zotmov_preferences.png
    └── PackageDevelopment
        ├── GroupMeeting_packaging_09032026.pdf
        ├── How2Package.css
        └── How2Package installation workshop.html
```

- (d) The `lt` output is a combination of both `ls` and `tree` to display the hierarchical structure of a folder

Figure 3.3.: Visual representation of my customised `ls` commands to display the content of a folder from a Bash terminal.

3.6. Pimp your shell: switch from boring Bash to fancy Zsh

Listing 3.6 Install eza from the community APT repository

```
# install the gpg command
sudo apt update
sudo apt install -y gpg

# install eza
wget -qO- https://raw.githubusercontent.com/eza-community/eza/main/deb.asc | sudo gpg --dearmor -o /etc/apt/keyrings/gierens.gpg
echo "deb [signed-by=/etc/apt/keyrings/gierens.gpg] http://deb.gierens.de stable main" | sudo tee /etc/apt/sources.list.d/gierens.list
sudo chmod 644 /etc/apt/keyrings/gierens.gpg /etc/apt/sources.list.d/gierens.list
sudo apt update
sudo apt install -y eza
```

4. Software to install

General recommendation: avoid using ChatGPT for installation steps. Instructions are often deprecated and/or do not follow Ubuntu best practice.

i Note 1: Install software on Ubuntu: guidelines

Three recommended methods, depending on Ubuntu's support:

1. If supported by Ubuntu, prefer `apt install` for system software¹:
 - *Automatic updates* and security patches — **the only method that provides this systematically.**
 - Easiest installation method.
 - Signed and verified packages.
 - Recommended for *core system*, long-term software.
 - If **not directly available** in Ubuntu's native package manager, or if the back end uses `snap`, switch to a pre-built tarball², or add a repository (only if **trusted**) to the `apt` keyrings. Examples: Docker (Section 4.5), or the custom Zotero installation script.
 - **.deb files** target Debian-derived distributions (Ubuntu, Linux Mint, etc.); **.rpm files** target Red Hat-based systems (Fedora, CentOS, RHEL) and openSUSE:

4. Software to install

- No auto-updates from the vendor archive alone.
 - Potential dependency issues.
 - **Always install with `sudo apt install ./package.deb`, rather than `sudo dpkg -i`, to resolve dependencies, fix broken installs, and keep the system consistent.**
-

2. AppImages are standalone executable files that bundle the application and most of its dependencies:

- Recommended for *desktop applications* installed only for the current user — usually no `sudo` rights required.
 - Portable.
 - Store them under `~/Applications` and add a desktop entry if you want them in the application menu. Prefer AppImages over Snap for most desktop apps.
-

3. Pre-built tarball (`.tar.gz` / `.tar.xz`) archives:

- Stronger customisation.
 - Work without distribution packages.
 - Require manual handling and have no automatic updates.
 - **Only recommended for developers, or when nothing else exists.**
-

Least recommended options:

- `snap` stores pre-compiled tools, but is less suitable than `sudo apt install` for most core software.

- Flatpak + Flathub (acceptable when no better option exists; see Bitwarden, Section 4.16).

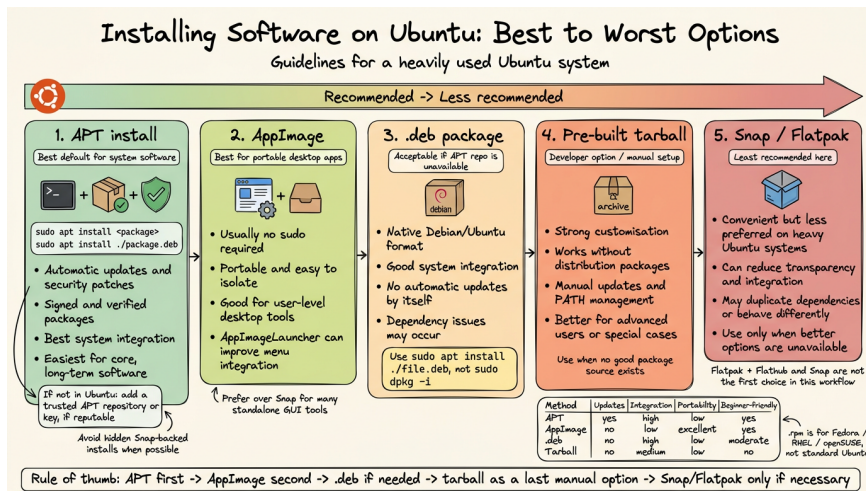


Figure 4.1.: Guidelines for installing software on Ubuntu

Portable versus non-portable: portable installs are easier to remove (fewer leftovers in the core system) but usually lack automatic updates. Non-portable applications are often better for daily or intensive use, but are not sandboxed and are harder to remove cleanly.

²apt-get works similarly, but is now considered legacy for interactive use; prefer apt.

²Fetch the latest stable archive, extract it, and add executables to your PATH; see Section 4.10.

4. Software to install

4.1. Git and GitHub configuration

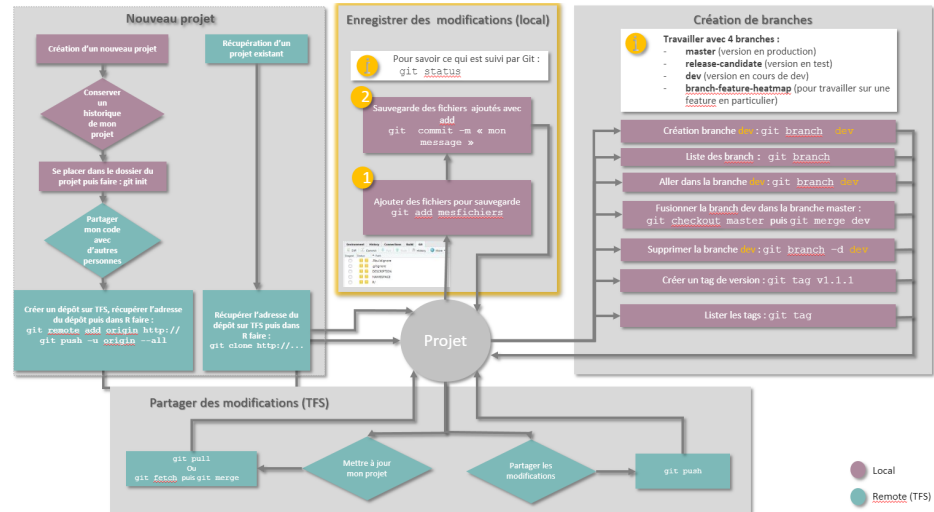


Figure 4.2.: Overview of recommended GitHub security and collaboration practices

1. GitHub often requires two-factor authentication; set it under <https://github.com/settings/security> (see also Figure 4.2). Prefer a *passkey* if your device supports it (Linux often does not), or an authenticator app such as Microsoft Authenticator. **Avoid SMS + mobile phone as the sole second factor.**
2. Configure your access to GitHub:
 - i. Choose either SSH or HTTPS.
 - ii. For HTTPS, create one token per device under <https://github.com/settings/tokens>. Name the token after your laptop, its configuration, and owner. Assign at least the repo and related read scopes. Copy it once — **it will not be shown again.**

3. General options to configure:

- i. Use the command line (this also creates `~/.gitconfig`):

Listing 4.1 Set global Git identity

```
git config --global user.name "<your-github-username>"
git config --global user.email "<your.email@example.org>"
```

- ii. Or edit `~/.gitconfig` directly (more prone to indentation errors). A default configuration is shown in Listing 4.2; for a fuller description of options, see Formation Git 2024, a tutorial by Benjamin:

4. Avoid typing your password for every login:

- i. For HTTPS + personal access token (PAT), use `git config --global credential.helper store` for **permanent** storage of the credential (consider whether the on-disk store should be encrypted), or `git config --global credential.helper 'cache --timeout=3600'` for temporary access³.
- ii. Or define an SSH key and use the SSH remote URL.

4.2. PDF management

4.2.1. Sioyek as PDF viewer

Sioyek is an open-source PDF viewer aimed at technical papers with mathematical formulae. Typical uses include coloured *highlights* (h), smart jumps to figures by clicking them, and bookmarks (b).

1. Choose a portable build or a standard distribution from the Sioyek releases.

³Time-out units are seconds; 3600 is one hour.

4. Software to install

2. Unzip the archive and make the binary executable with `chmod +x`.
3. Run it.

i From archive to PATH and Start menu: a step-by-step Ubuntu example

To turn an executable into a **desktop application** on Ubuntu GNOME, follow these steps (paths follow Sioyek; other packages may differ):

1. Relocate files under **/opt/sioyek**⁴.

Listing 4.3 Install Sioyek from archive

```
## create required folders
sudo mkdir -p /opt/sioyek
sudo mkdir -p /etc/sioyek
sudo mkdir -p /usr/share/sioyek

## relocate, if not already done, the sioyek builds

### core executables
sudo cp ~/sioyek/build/sioyek /opt/sioyek/
sudo chmod +x /opt/sioyek/sioyek

### configuration files
sudo cp ~/sioyek/build/prefs.config /etc/sioyek/
sudo cp ~/sioyek/build/keys.config /etc/sioyek/
sudo cp -r ~/sioyek/build/shaders /usr/share/sioyek/
sudo cp ~/sioyek/build/tutorial.pdf /usr/share/sioyek/
```

-
2. Add an icon and a GNOME launcher entry:

Listing 4.4 Sioyek desktop entry

```
sudo mkdir -p /usr/share/icons/hicolor/256x256/apps
## place sioyek.png in that folder (downloaded from the web)

sudo tee /usr/share/applications/sioyek.desktop > /dev/null <<'EOF'
[Desktop Entry]
Version=1.0
Type=Application
Name=Sioyek
GenericName=PDF Viewer
Comment=Fast keyboard-driven PDF viewer
Exec=/opt/sioyek/sioyek %f
Icon=sioyek
Terminal=false
Categories=Office;Viewer;
MimeType=application/pdf;
StartupNotify=true
EOF

## Refresh desktop database
sudo update-desktop-database
```

3. (Optional) Make Sioyek the default **PDF viewer**, using either:

- a. GUI method:
 1. Right-click any PDF.
 2. **Properties** → **Open With**.
 3. Select **Sioyek**.
 4. Click **Set as default**.
- b. Or the command line:

4. Software to install

Listing 4.5 Set Sioyek as default PDF viewer

```
xdg-mime default sioyek.desktop application/pdf

## to check default
xdg-mime query default application/pdf
```

4. Add Sioyek to the PATH via a symbolic link:

Listing 4.6 Symlink Sioyek into PATH

```
sudo ln -s /opt/sioyek/sioyek /usr/local/bin/sioyek
```

4.2.2. Sejda for interactive PDF editing

- Web version
- Sejda *add-on* for Chrome

4.2.3. PDFtk

- Use PDFtk if you prefer editing PDFs from the command line (`sudo apt install pdftk` when available).

⁴/opt is the usual location for third-party apps, which helps avoid conflicts with distribution packages.

4.3. Image editors (to replace Paint on Windows)

Sorted from closest experience to Paint, to most generalist / specialised:

1. Drawing — lightweight local Paint-like editor:
 - i. Add Drawing to the apt repository: `sudo add-apt-repository ppa:cartes/drawing`
 - ii. Install with `sudo apt install drawing`
2. LibreOffice Draw — diagrams and simple page layouts.
3. **Photopea** — preferred **free**, browser-based Photoshop alternative when you need layers, masks, PSD/PSB import, and a large toolset without installing desktop software. (Pixlr / “PhotoEditor”-style tools are fine for quick crops, but Photopea is more versatile in free mode and closer to a full raster editor.)
4. GIMP (desktop equivalent to a large part of Photoshop’s scope):
5. Inkscape for **vector** graphics and fully editable icons:
6. Background removal for figures / slides: use an online tool such as remove-bg.io when you need a clean cut-out of a subject (for example an organism or device photo) before composing the panel in Photopea, GIMP, or Inkscape. Prefer this for outreach slides; keep raw scientific image panels under version control without destructive edits when possible.

4.4. LibreOffice

1. Install the main suite (Writer, Calc, ...) with:
2. Useful extensions (add via **Tools** → **Extensions**):
 - i. Rotate images in Writer with the *Rotate* extension.

4. Software to install

- ii. Install dictionaries and grammar tools (Grammacleete and LanguageTool). *Check whether LanguageTool is still worth the resource cost; it is AI-based and can be heavy.* You may configure the Java version under **Tools** → **Options** → **Advanced**. Also install the French localisation package:
 - iii. Install the TeXMaths LaTeX extension.
3. Font customisation:
- i. Install core Windows fonts:
 - ii. Extra fonts (or use the native Fonts manager under GNOME). Many popular fonts are available under the Every-Font GitHub repository:

Core typographic options

The main stylistic variants of fonts you can control are:

1. Weight (Light, Regular, Bold, ...)
2. Angle or shape (regular versus italic)
3. Width
4. Serif versus sans-serif. This property is structural to a typeface; for example, Times New Roman is serif, and Century Gothic is sans-serif. **For data visualisations or presentations, sans-serif typefaces are usually preferred; the reverse often holds for printed documents.**
5. Advanced options (ligatures, etc.) under **LibreOffice** → **Format** → **Character** → **Features** (see Figure 4.3).

Make sure, if available, to download at least the regular, bold, italic, and bold-italic versions of each font.

Key Typographic Options:

Customizing the Shape of a Font

1. Weight

Thickness of the Strokes.

Thin

Light

Regular

Medium

Bold

Black

2. Style (Posture)

Angle or Shape of the Letters.

Courier Gothic Roman

Courier Gothic *Italic*

Courier Gothic *Oblique*

3. Width

Horizontal Proportion.

Condensed

Normal

Expanded

4. Serif vs. Sans-Serif

Serif	Cens-Serif
Times New Roman	Century Gothic

5. Small Caps

Smaller Uppercase Letters.

COURIER GOTHIC SMALL CAPS

6. OpenType Features

- Ligatures *fi fl*
- Old-Style Numerals **123**
- Tabular Numbers **123**
- Stylistic Sets **a a**

*These can be accessed in LibreOffice - Format
→ Character - Features.*

Figure 4.3.: Main typographic options

4. Software to install

Listing 4.2 Example ~/.gitconfig

```
[user]
  name = <your-github-username>
  email = <your.email@example.org>
## modern default git options
[init]
  defaultBranch = main

## simplifying solving git conflicts + troubleshoot solving
[merge]
  conflictstyle = zdiff3
[rerere]
  enabled = true

## tolerate distinct end of lines
[core]
  autocrlf = input

## visualisation
[tag]
  sort = version:refname
[branch]
  sort = -committerdate
[credential]
  helper = store
```

Listing 4.7 Install GIMP

```
sudo apt install gimp
sudo apt install gimp-plugin-registry ## add extensions
```

Listing 4.8 Install Inkscape

```
sudo apt update
sudo apt install inkscape
```

Listing 4.9 Install LibreOffice

```
sudo apt install libreoffice libreoffice-gtk3
```

Listing 4.10 French dictionary for LibreOffice

```
## French dictionary / hyphenation
sudo apt update
sudo apt install libreoffice-l10n-fr
```

Listing 4.11 Install Microsoft core fonts

```
sudo apt update
sudo apt install ttf-mscorefonts-installer ## Microsoft core fonts
fc-cache -f -v ## rebuild font cache so fonts appear in LibreOffice
```

Listing 4.12 Install extra fonts

```
mkdir -p ~/.local/share/fonts ## ensure personal fonts folder exists
wget -P ~/.local/share/fonts \
    "https://raw.githubusercontent.com/serendipious/every-font/master/CenturyGothic-Italic%20-%20Century
fc-cache -f -v ## update font cache
```

4. Software to install

4.5. Docker

1. Download dependencies, configuration files, and add the Docker repository to `apt` for automatic updates (Listing 4.13):

Listing 4.13 Add Docker APT repository

```
sudo apt update
sudo apt install ca-certificates curl

## Part I: create, download and make Docker apt key readable
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
  -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

## Part II: define where and which Docker packages to install
sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Signed-By: /etc/apt/keyrings/docker.asc
EOF

## Part III: refresh package lists
sudo apt update
```

2. Install the Docker toolkit:
3. Run Docker without `sudo` for your main laptop user (Listing 4.15):

Listing 4.14 Install Docker Engine

```
sudo apt install docker-ce docker-ce-cli containerd.io \
docker-buildx-plugin docker-compose-plugin
```

Listing 4.15 Allow Docker without sudo

```
# create a Unix group allowing Docker without admin rights
sudo groupadd docker
# add the current user to it
sudo usermod -aG docker $USER
```

4. Check the installation with `sudo docker run hello-world`. If Docker does not start on login, enable it with `sudo systemctl enable --now docker`. More detail is in the official Ubuntu install guide.

Useful Docker tutorials:

- Benjamin’s presentation
- VS Code, R and Docker with Dev Containers
- VS Code, Python and Docker with Dev Containers
- Dev Containers, from INRIA trainings

4.6. Web browsers

4.6.1. Firefox

Avoid `sudo apt install firefox` without first configuring Mozilla’s APT source — the default Ubuntu package may pull in `snappy` and a Snap-based Firefox.

4. Software to install

1. Import and add the Mozilla APT signing key (Listing 4.16):

Listing 4.16 Add Mozilla Firefox APT key

```
## Download APT key
wget -q https://packages.mozilla.org/apt/repo-signing-key.gpg -O- \
| sudo tee /etc/apt/keyrings/packages.mozilla.org.asc > /dev/null

## Add it to the Apt registry
cat <<EOF | sudo tee /etc/apt/sources.list.d/mozilla.sources
Types: deb
URIs: https://packages.mozilla.org/apt
Suites: mozilla
Components: main
Signed-By: /etc/apt/keyrings/packages.mozilla.org.asc
EOF

## Prefer Mozilla Firefox over Snap packaging
echo '
Package: *
Pin: origin packages.mozilla.org
Pin-Priority: 1000
' | sudo tee /etc/apt/preferences.d/mozilla
```

2. Install Firefox from Mozilla:

4.6.2. Google Chrome

1. Download the .deb archive, then install it:
2. Add Google's APT signing key for automatic updates (Listing 4.19):

Listing 4.17 Install Firefox from Mozilla APT

```
sudo apt update
sudo apt install firefox

which firefox ## should not report a snap path
apt policy firefox
```

Listing 4.18 Download Google Chrome .deb

```
sudo apt install ./google-chrome-stable_current_amd64.deb
```

3. Optionally add the *Grammarly* Chrome plug-in for spelling and grammar checks (pick one regional English variety and stick to it — British English for this course).

4.7. Slack

1. Download the Slack application.
2. Install with `sudo apt install ./slack-desktop-4.47.69-amd64.deb` (adjust the version string to match the file you downloaded).
3. Update later with:

4.8. Zulip

1. Install from Zulip’s APT repository (Listing 4.21):
2. Log in (GitHub profile is convenient) on each organisation or channel you need, so you stay notified of new posts.

4. Software to install

Listing 4.19 Install Google Chrome

```
## Download APT key
wget -q -O - https://dl.google.com/linux/linux_signing_key.pub \
| sudo gpg --dearmor -o /usr/share/keyrings/google-linux-signing-key.gpg

## Add it to the Apt registry
echo "deb [arch=amd64 signed-by=/usr/share/keyrings/google-linux-signing-key.gpg] ht
| sudo tee /etc/apt/sources.list.d/google-chrome.list

## Update package lists
sudo apt update
```

Listing 4.20 Upgrade Slack

```
sudo apt-get update
sudo apt-get upgrade slack-desktop
```

4.9. Cytoscape

Major caveat: Cytoscape is only supported on Java 17 (not on newer stable Java versions).

1. Install Java 17 with `sudo apt install openjdk-17-jdk`⁵.
2. Install Cytoscape from the vendor installer (not Docker):

⁵On many Ubuntu installs a newer Java (e.g. 21) is already present; keep Java 17 available for Cytoscape.

- i. Retrieve the latest version.
- ii. Make the installer executable with `chmod u+x <filename.sh>`.
- iii. Run with `bash <filename.sh>`.
- iv. Launch with Cytoscape &.

Listing 4.21 Install Zulip desktop

```

sudo apt install curl ## if not already installed
sudo curl -fL -o /etc/apt/trusted.gpg.d/zulip-desktop.asc \
    https://download.zulip.com/desktop/apt/zulip-desktop.asc
echo "deb https://download.zulip.com/desktop/apt stable main" \
    | sudo tee /etc/apt/sources.list.d/zulip-desktop.list
sudo apt update
sudo apt install zulip

```

4.10. Zotero

1. Install Java support for the LibreOffice plug-in:

Listing 4.22 Ensure Java for Zotero

```

sudo apt install libreoffice-java-common -y

```

2. Retrieve the latest stable build from <https://www.zotero.org/download/>⁶.
3. Extract and link the executable (Listing 4.23), then install the Zotero Connector in your browser:
4. (Optional) Create a desktop launcher at `~/.local/share/applications/zotero.desktop` with the content in Listing 4.24, then make it executable: `chmod +x ~/.local/share/applications/zotero.desktop`.
5. Inspired by the Zotero hacks tutorial, keep article files synced across machines while controlling what Zotero stores in the cloud:

⁶Alternatively, download the archive with `curl`.

4. Software to install

Listing 4.23 Extract and link Zotero

```
#!/bin/bash

## part I: extract archive
sudo tar -xjf zotero.tar.bz2 -C /opt
sudo mv /opt/Zotero_linux-x86_64 /opt/zotero

## Part II: add the executable to PATH
sudo ln -s /opt/zotero/zotero /usr/local/bin/zotero
```

- i. Add the plug-ins **Better BibTeX** (needed for exporting a clean .bib file) and **Zotmoov** (replaces the older ZotFile extension for filing PDFs by author name).
- ii. Edit Zotero preferences⁷:
 - Uncheck automatic web-page snapshots (reduces clutter from many small files).
 - Uncheck full-text sync.
 - Under **Edit** → **Settings** → **Advanced**, keep separate:
 - the **base directory** (the folder synced by your file sharing software), and
 - a **custom data directory** in a different location, managed by Zotero itself (Figure 4.4).

iii. Configure **Better BibTeX** citation keys the same way on every machine (for example
`[auth:lower][year][journal:lower:abbr]`).

<!-- -->

⁷Uniformity across platforms matters more than any single absolute path.

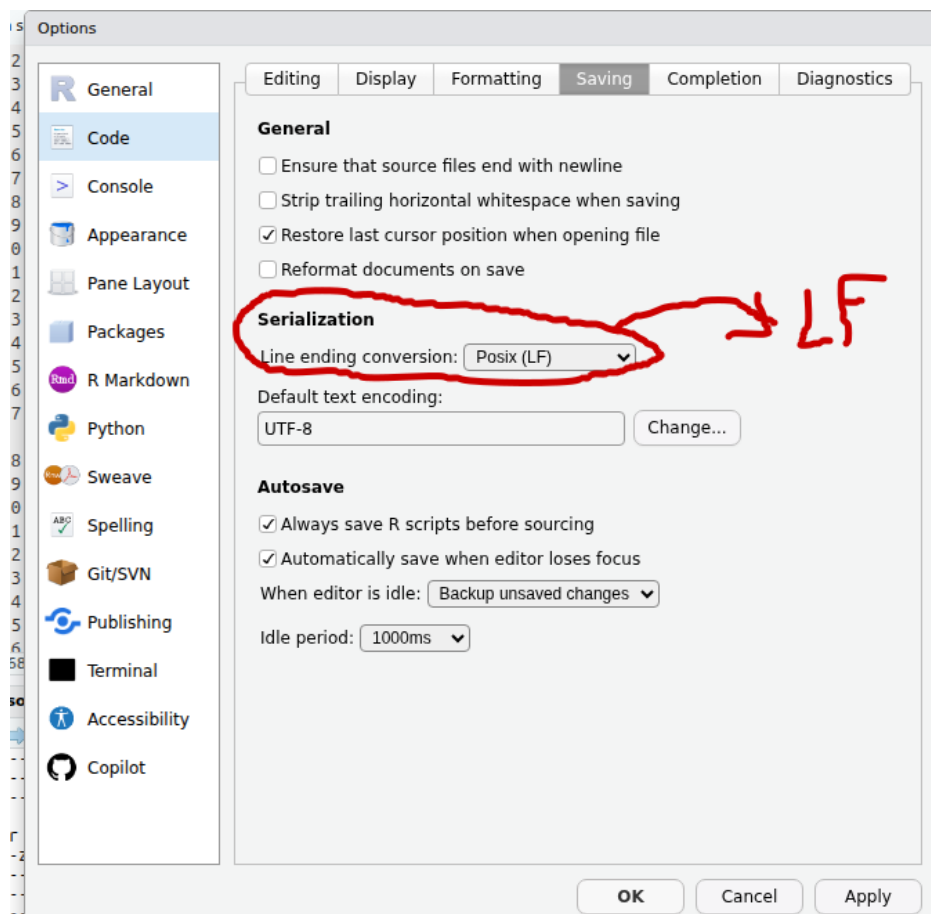


Figure 4.4.: Zotero advanced database configuration

4. Software to install

Listing 4.24 Zotero desktop entry

```
[Desktop Entry]
Name=Zotero
Exec=/usr/local/bin/zotero
Icon=/opt/zotero/icons/icon128.png
StartupWMClass=Zotero
Type=Application
Terminal=false
Categories=Office;
MimeType=text/plain;x-scheme-handler/zotero;application/x-research-info-systems;text/xml;
X-GNOME-SingleWindow=true
```

4.11. Quarto, R and Cursor

4.11.1. R and recommended packages

1. To get recent R builds, add the CRAN signing key and repository (Listing 4.25). Adjust the Ubuntu codename (jammy, noble, ...) to match your release:

Listing 4.25 Add CRAN APT repository for R

```
sudo mkdir -p /etc/apt/keyrings
sudo wget -O /etc/apt/keyrings/cran.gpg \
  https://cloud.r-project.org/bin/linux/ubuntu/marutter_pubkey.asc
echo "deb [signed-by=/etc/apt/keyrings/cran.gpg] https://cloud.r-project.org/bin/linux/ubuntu/
| sudo tee /etc/apt/sources.list.d/cran.list
```

2. Install R:
 - i. Install base packages:

Listing 4.26 Install R base

```

sudo apt update
sudo apt install r-base r-base-dev -y
R --version

```

- ii. Install recommended Ubuntu system libraries for scientific computing and network/plotting support (@lst-r-syslibs):

```
<!-- -->
```

- 3. Configure a default CRAN mirror:

- i. The official mirror <https://cloud.r-project.org> is the most up to date.
- ii. On Ubuntu, the **Posit Package Manager** mirror is often much faster because it serves prebuilt Linux binaries. Pin it in `~/.Rprofile` (Listing 4.28), replacing `<ubuntu-version>` with your release (e.g. `noble`):

```
<!-- -->
```

- 4. Install packages in a reproducible way (project lockfiles, `renv`, or a curated meta-package). Stephen Turner's tidyverse-oriented stack is a useful starting set for many bioinformatics pipelines: `remotes::install_github("stephenturner/Tverse", upgrade = TRUE)`.
- 5. Install Air, the recommended fast formatter for R, as a user-level CLI tool via `uv` (Listing 4.29). Configure Cursor (or your editor) to call `air` on save; see the Air documentation for editor integration.

4. Software to install

4.11.2. Quarto

Install Quarto from the official site: <https://quarto.org/docs/get-started/>.
Example local `.deb` install (Listing 4.30):

A minimal inline maths check:

$$a = 10$$

4.11.3. Cursor with R

1. Install the following R packages for language support, rendering, and debugging (Listing 4.31):
2. (Optional but recommended) Use `radian` as an enhanced R terminal:
3. Install the `vscDebugger` / R debugger extension in Cursor, then run the debug demo script to test it (Figure 4.5).

4.12. Python

From **PEP 668**, on modern Debian/Ubuntu with a system Python of 3.11 or newer, `pip` is blocked from modifying the **system Python** so that it cannot break `apt`-managed packages.

This section evolves quickly. Prefer the `uv`-centric workflow below; treat older `pip` / Poetry / Conda advice as legacy unless you are constrained by an HPC site.

R Debugger — RStudio vs VS Code / Cursor		
Five shared actions, one keyboard shortcut each		
RStudio	Action	VS Code / Cursor
1 — STEP OVER ↩ + n Next	Step Over <code>result <- my_func(x) # ← steps OVER my_func</code>	↩ + F10 Step Over
2 — STEP INTO ↪ + s Step Into	Step Into <code>result <- my_func(x) # ← steps INTO my_func</code>	↓ + F11 Step Into
3 — STEP OUT / FINISH ⬅ + f Finish	Step Out <code># inside my_func() body # ← exits back to caller</code>	↑ + Shift+F11 Step Out
4 — CONTINUE ▶ + c Continue	Continue <code>browser() # pause 1 # ... code ... browser() # pause 2 ← jumps here</code>	⏮ + F5 Continue
5 — STOP ■ + q Stop	Stop / Quit <code># exits debug mode # returns to normal R session</code>	■ + Shift+F5 Stop
ⓘ VS Code only — Reload Sources (⇧): reloads the file after edits without restarting the session Edit debug_demo.R → click ⇧ → breakpoints update immediately		

Figure 4.5.: Side-by-side comparison of the debug demo script and the debugger in action

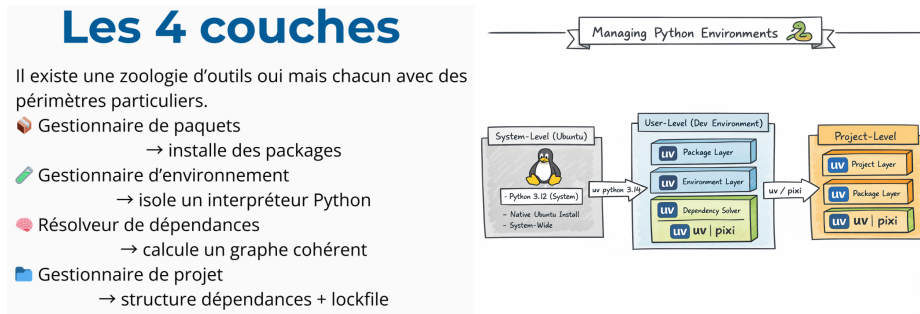
4. Software to install

4.12.1. Modern Python guidelines

💡 Modern Python development guidelines

1. Recent Ubuntu releases (including Noble) strongly discourage installing extra system-wide Python interpreters. Avoid `sudo apt install python3.14-full` and Deadsnakes PPAs for day-to-day work.
2. **uv replaces** a cluster of slower, single-purpose tools (`pip`, `poetry` for many workflows, `virtualenv`, `pipx`, ...) with a faster unified CLI — see Figure 4.7 and Figure 4.6. Poetry may still be preferable for some package-publishing edge cases; `pip` remains the legacy default on older HPC images.
3. For **multi-language** environments, `pixi` has largely superseded Conda/Mamba. Use **uv for Python-centric projects**; use `pixi` when you need non-Python dependencies in the same lock-file. Note that `uv` enforces virtual environments when running project scripts — see Table ??.
4. The ecosystem moves fast: reassess tooling yearly.

Feature	uv	Poetry	Pixi
Speed			
Dependency management			
Virtual environments			
Python version install			
Package publishing	Acceptable	Best	Limited
Non-Python dependencies			
CI performance			
Maturity	Medium	High	Medium



- (a) Each tool used to have its own perimeter, hence the diversity of Python environments, from system-wide (do not modify) to the project level

	Paquets	Env	Resolver	Projet
pip	✓	✗	✓	✗
venv	✗	✓	✗	✗
conda	✓	✓	✓	✗
mamba	✓	✓	✓ (rapide)	✗
poetry	✓	✓	✓	✓
uv	✓	✓	✓ (rapide)	✓
pixi	✓	✓	✓	✓

- (c) Side-by-side comparison of core use cases for each Python tool — uv covers most of them

Figure 4.6.: Comparison of Python managers and environment layers.

4. Software to install



- (a) Main commands that replace older Python tools with uv
- (b) Caricature of the recent dominance of uv
- (c) Decision tree: uv for pure Python projects, Poetry (or uv) for packaging edge cases, and pixi for cross-language projects. Conda and Mamba are no longer required in a modern default stack.

Figure 4.7.: uv as the default Python tooling layer.

4.12.2. One tool to bind them: a uv workflow

The steps below install a recent Python (example: 3.14) **without** replacing the system interpreter Ubuntu depends on.

1. Install uv:
2. Install a managed Python with uv:
3. Pin the Python version at one of two levels:
 - i. Project level: `uv python pin 3.14`.
 - ii. User level:

<!-- -->

4. Prefer `uv tool install` (instead of `pipx`) for CLI utilities — formatters, notebook tools, hooks, and so on. As with `pipx`, these tools are added to the user `PATH` (Listing 4.36):

Python version management: core checks

- Check the system Python:

Listing 4.37 Inspect system Python

```
ls /usr/bin/python* ## typically a single 3.12 on modern Ubuntu
```

- Check versions managed by uv:

4. Software to install

Listing 4.38 Inspect uv-managed Python

```
ls -la ~/.local/share/uv/python*  
## paths should live under your HOME directory
```

Checking which Python is active:

- From the terminal (system-wide), `ls -d /usr/lib/python3.*` or `which -a python` should point at the OS Python only when you intend to use it.
- From a Python shell (or Marimo / optional Jupyter):

Listing 4.39 Python environment summary

```
import sys  
print(sys.version)  
print(sys.executable)  
print(sys.path)
```

Useful reading:

- From Pip to Lightning: How uv Opened My Eyes to Better Python Workflows
- LinkedIn post on Python dependency managers

4.12.3. Marimo (preferred over Jupyter Lab)

Marimo is the recommended interactive notebook tool for this course: notebooks are plain Python files, reactive, and easy to version-control. Jupyter Lab remains available as an optional `uv tool` if a collaborator requires it.

1. Install Marimo with commonly used extras (DuckDB SQL cells, Polars / PyArrow, Ruff, Vegafusion charts):
2. Add the Cursor / VS Code extension:
3. Suggested keybindings (add to `keybindings.json`):
4. An interactive example is provided in `demo.py`.

4.13. Nextflow

Install Nextflow as a standalone, self-updating binary. Confirm that Java 17 is available first (see Section 4.9):

4.14. LaTeX

4.15. Cursor

1. Download the `.deb` package.
2. Install it (Listing 4.45):
3. Sync your Cursor configuration across devices — see

4. Software to install

1.

Tip 1: Export Cursor settings

1. Export the core settings:
 - i. Open the Command Palette (Ctrl/Cmd + Shift + P), then search for **Preferences: Open Settings (JSON)**.
 - ii. Copy that file onto the new device.
 - iii. Default Linux path: `~/.config/Cursor/User/settings.json`.
2. Export keyboard shortcuts:
 - i. Open **Preferences: Open Keyboard Shortcuts (JSON)**.
 - ii. Copy `keybindings.json` to `~/.config/Cursor/User/keybindings.json` on the new machine.
3. (Main difference from VS Code.) Export the extension list manually:
 - i. List extensions with `cursor --list-extensions > extensions.txt`. To also record each extension's version, use `cursor --list-extensions --show-versions > extensions.txt`.
 - ii. Reinstall them with Listing 4.46:

Listing 4.46 Reinstall Cursor extensions

```
grep -v '^s*#' ~/.config/Cursor/User/extensions.txt \  
| grep -v '^s*$' \  
| xargs -I{} cursor --install-extension {}
```

```
<!-- -->
```

More detail (including symlink automation) is in *Sync Cursor Settings the Dotfiles Way*.

For syncing configuration across many users, see also Benjamin's notes and Cursor Config Sync.

4.16. Password manager: Bitwarden

Bitwarden is the recommended password manager on Linux for most teaching and research setups: it is largely free, works with major browsers (Firefox, Chrome, ...), syncs across many devices, and stores more than passwords (passkeys, SSH secrets, payment cards). Lightweight mobile apps exist for iPhone and Android. It also covers Windows and Linux, which goes well beyond the local GNOME **Passwords and Keys** tool.

One limitation: it does not automatically flag duplicate entries as thoroughly as some commercial managers.

Follow the steps in Listing 4.47⁸.

In the **Auto-fill** settings, leave **Do not auto-fill on page load** unticked, and set **Default URI matching** to **Exact** to reduce ambiguous fills. Counter-intuitively, use the Bitwarden Web Vault for bulk deletion of entries; that workflow is not fully exposed in the desktop app.

⁸Contrary to the preference for `apt` in Note 1, only Flatpak and Snap installs are practical for Bitwarden on Ubuntu — so you do not get the same native `apt` update story.

4. Software to install

Listing 4.27 System libraries for R packages

```
## Compilation tools
sudo apt install cmake

## Network, imaging, and plotting libraries
sudo apt install \
  build-essential \
  libpng-dev \
  libjpeg-dev \
  libtiff-dev \
  libcairo2-dev \
  libcurl4-openssl-dev \
  libssl-dev \
  libxml2-dev \
  zlib1g-dev

## Scientific libraries
sudo apt install \
  build-essential \
  gfortran \
  libblas-dev \
  liblapack-dev \
  libgsl-dev
```

Listing 4.28 Posit Package Manager in ~/.Rprofile

```
echo 'options(repos = c(CRAN = "https://packagemanager.posit.co/cran/__linux__/<ubuntu__>
>> ~/.Rprofile
```

Listing 4.29 Install Air formatter via uv

```
uv tool install air-formatter
```

Listing 4.30 Install Quarto

```
sudo apt install ./quarto-1.3.496-linux-amd64.deb
quarto check
```

Listing 4.31 R packages for Cursor debugging

```
install.packages(c(
  "remotes",
  "languageserver",
  "rmarkdown",
  "httpgd"
))
remotes::install_github("ManuelHentschel/vscDebugger")
```

Listing 4.32 Install radian via uv

```
uv tool install radian
```

Listing 4.33 Install uv

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Listing 4.34 Install a Python version with uv

```
uv python install 3.14
```

4. Software to install

Listing 4.35 User-level Python with uv

```
cd ~  
echo "3.14" > ~/.python-version
```

Listing 4.36 Install CLI tools with uv tool

```
uv tool install ruff  
uv tool install "dvc[ssh]" # data versioning when Git LFS is not enough  
uv tool install black  
uv tool install marimo      # preferred notebook / reactive Python UI  
uv tool install ipython  
uv tool install pre-commit  
uv tool install cookiecutter # project templates for packaging  
## optional legacy / niche tools:  
# uv tool install jupyterlab  
# uv tool install poetry
```

Listing 4.40 Install Marimo via uv

```
uv tool install "marimo[recommended]"  
  
# Verify  
marimo --version
```

Listing 4.41 Install Marimo Cursor extension

```
cursor --install-extension marimo-team.vscode-marimo
```

Listing 4.42 Suggested Marimo keybindings

```
// --- Marimo ---
// Open current .py file as a marimo notebook in the editor
{
  "key": "ctrl+alt+m",
  "command": "marimo.openNotebook",
  "when": "resourceExtname == '.py'"
},
// Create a new sandboxed marimo notebook via terminal
{
  "key": "ctrl+alt+shift+m",
  "command": "workbench.action.terminal.sendSequence",
  "args": { "text": "uvx marimo edit --sandbox \n" }
},
// Run current marimo notebook as a read-only app
{
  "key": "ctrl+alt+r",
  "command": "workbench.action.terminal.sendSequence",
  "when": "resourceExtname == '.py'",
  "args": { "text": "uvx marimo run '${file}'\n" }
}
```

Listing 4.43 Install Nextflow

```
curl -s https://get.nextflow.io | bash
chmod +x nextflow

# move nextflow onto the user PATH
mkdir -p "$HOME/.local/bin"
mv nextflow "$HOME/.local/bin/"
```

4. Software to install

Listing 4.44 Install TeX Live packages

```
sudo apt install texlive-full -y
## commonly used extras (often already covered by texlive-full)
sudo apt install texlive-latex-extra texlive-fonts-recommended \
texlive-science biber latexmk -y
```

Listing 4.45 Install Cursor

```
sudo apt install ./cursor_2.0.77_amd64.deb
```

Listing 4.47 Install Bitwarden via Flatpak

```
## Install Flatpak and the GNOME Flatpak plugin
sudo apt install flatpak
sudo apt install gnome-software-plugin-flatpak

## Add the Flathub repository
flatpak remote-add --if-not-exists flathub \
https://dl.flathub.org/repo/flathub.flatpakrepo

## Install and run Bitwarden locally
flatpak install flathub com.bitwarden.desktop
flatpak run com.bitwarden.desktop

## Install the browser add-on (Chrome example), then pin it:
## https://chromewebstore.google.com/detail/bitwarden-password-manage/nngceckbapebfim

## Export, then import your password database from other browsers
```

5. AMU tools and remote servers

5.1. Core numerical tools provided by AMU

List of numerical tools used at AMU.

5.1.1. AMU VPN (GlobalProtect)

Staff and students often need the AMU VPN to reach internal services (clusters, library resources, some web apps) from outside the campus network. Official documentation:

- AMU VPN overview (DUD)
- Strong authentication (2FA) — enrol / confirm your second factor **before** relying on VPN from home

AMU currently ships **Palo Alto GlobalProtect**. On Linux, prefer the **graphical UI** package rather than the CLI-only build suggested in some internal notes: the tray icon makes connect / disconnect and portal URL errors much easier to diagnose.

1. Download the latest Linux archive from the shared AMU folder (GlobalProtect Linux builds on AMUbox). Filenames look like **PanGPLinux-6.3.3-c22.tgz** — always take the **newest** release available there.
2. Extract it locally:

5. AMU tools and remote servers

Listing 5.1 Extract GlobalProtect Linux archive

```
tar -xvf PanGPLinux-*.tgz
cd PanGPLinux-*    ## adjust to the extracted directory name
ls *.deb           ## you should see both CLI and UI packages
```

Listing 5.2 Install GlobalProtect UI .deb

```
sudo apt install ./GlobalProtect_UI_deb-*.deb
```

3. Install the **UI .deb** with **apt** so dependencies resolve cleanly (replace the version string with the file you downloaded):



If apt cannot find local dependencies

Fall back to `sudo dpkg -i ./GlobalProtect_UI_deb-*.deb` then
`sudo apt -f install` to finish broken dependencies.

4. Log out and back in (or reboot) if the panel icons do not appear. You should then see the GlobalProtect indicator(s) in the top bar. Open the UI, set the AMU portal / gateway as documented by DUD, and authenticate with your university account **plus** strong authentication when prompted.
5. Smoke-test by opening an internal URL that is blocked off-campus without VPN. If connection fails, check 2FA enrolment first, then reinstall / update from the AMUbox archive (versions move quickly).

5.1.2. AMUZoom

1. Retrieve the latest version, choosing the proper Linux distribution and OS architecture under Zoom downloads. Also available online. It is strongly recommended to install Zoom locally.
2. Run `sudo apt install ./zoom_amd64.deb`.

5.1. Core numerical tools provided by AMU

3. **Log in with your university email address using SSO.** The account provides unlimited meetings by default; use the domain `univ-amu-fr`.¹

Two settings matter when creating a Zoom meeting so that any external user with the password can log in and share their screen (see Figure 5.1):

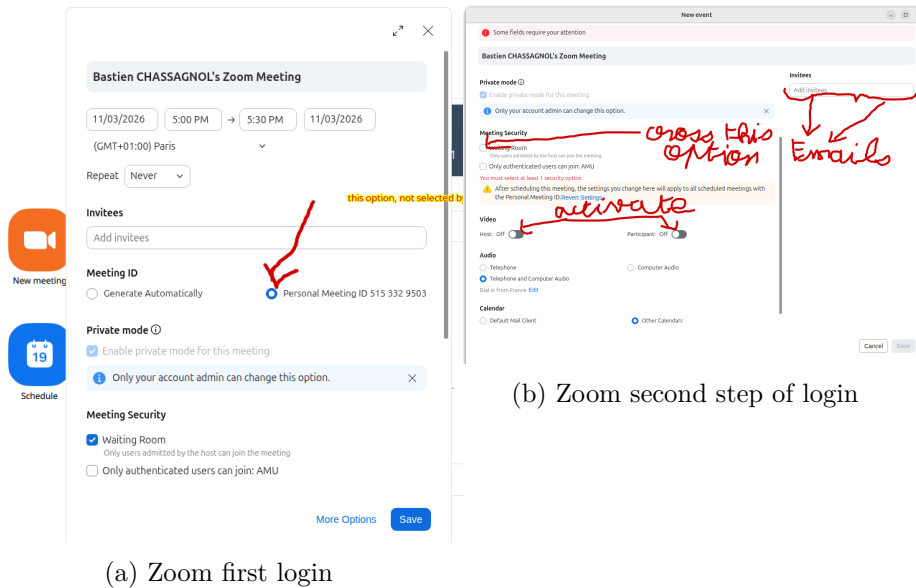


Figure 5.1.: Be careful to select the second, non-default option if you want to allow external users without an AMU email address to join your Zoom session.

Further resources:

- Guidelines for Zoom meeting creation and management
- Video tutorial
- Guidelines for students

¹Alternatively, use the web version available here.

5. AMU tools and remote servers

5.1.3. WooClap for QCMs

Go to WooClap and use your **institutional email address** for single sign-on across platforms.

5.2. Working with a remote server

Prefer **Cursor** or **VS Code** with the Remote SSH extension for everyday remote development: the editor runs locally while the terminal, language servers, and compute run on the cluster. Avoid relying on hosted desktop IDEs or notebook servers on the cluster for routine work.²

i Retrieve your identity on the server

Listing 5.3 server id

```
id ${USER}
# Example (local-style groups):
# uid=1001(bastien) gid=1001(bastien) groups=1001(bastien),4(adm),...
#
# Example on the MMG cluster (project group):
# uid=1007(<your-username>) gid=1007(<your-username>) groups=1007(<your-username>),...
```

Identity on a Linux server is defined by the combination of username, **UID** (user ID), and **GID** (group ID), plus membership of project groups. More detail is available in the Server Management tutorial.

²A practical split is: develop and test on small examples locally in Cursor, then scale on the server via Remote SSH or the command line.

5.2.1. SSH configuration

The dual aim is to avoid typing your password on every connection and to keep a safe, reusable list of remote hosts.

1. Set up SSH remote access (key-based authentication). The remote machine needs `openssh-server`:

Listing 5.4 openssh server

```
## remote server must have openssh-server installed
sudo apt install openssh-server
```

2. Generate an Ed25519 public-private key pair on your **local** machine:³

Listing 5.5 ssh keygen

```
ssh-keygen -t ed25519 -C "your_email@example.com"
```

This creates:

- `~/.ssh/id_ed25519` (private key)
 - `~/.ssh/id_ed25519.pub` (public key)
3. Copy your public key to the server with `ssh-copy-id username@server_ip`. After that, password login is no longer required for that host.
 4. (Optional) Keep a `~/.ssh/config` entry per cluster. Aliases such as `ssh ifb-core` then suffice, and the same file is what Cursor / VS Code Remote SSH uses to list hosts. Example:

³A passphrase is not compulsory, but it is safer if the disk is copied or stolen.

5. *AMU tools and remote servers*

A fuller host configuration (including default X11 forwarding) is available under `config-ssh`.

SSH configuration recommendations

On older Ubuntu setups, `ssh-agent` / `ssh-add` may not start by default. Enable the user service with `systemctl --user enable --now ssh-agent`.

See Table ?? and Figure 5.2 for a concise checklist and an overview of the first connection.

5.2. Working with a remote server

Practice	Why
Prefer Ed25519 keys	Small, fast, modern default.
Use a passphrase on the key	Protects the key if the disk is copied or stolen.
<code>~/.ssh/config</code> per host	Aliases and login options in one place.
Never commit private keys	Keep <code>id_*</code> out of Git and cloud-synced folders.
<code>ssh-copy-id</code> once per host	Standard way to populate <code>authorized_keys</code> .

5. AMU tools and remote servers

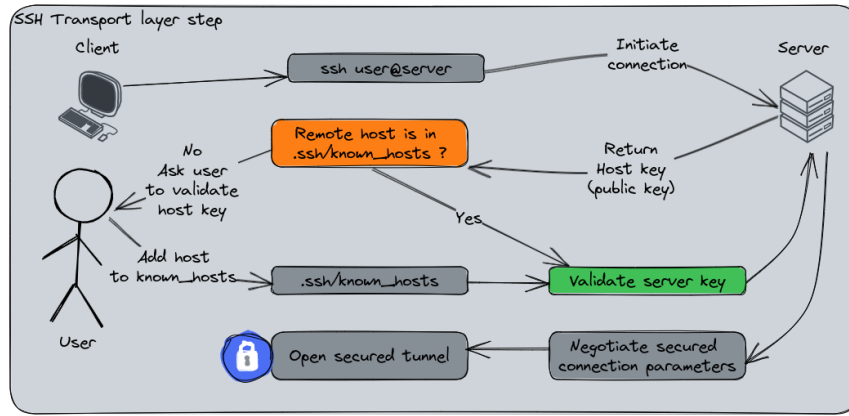


Figure 5.2.: Overview of an initial SSH connection

Option	Purpose	Effect on X11
-X	X11 forwarding (untrusted)	Runs GUI apps remotely
-Y	X11 forwarding (trusted)	Runs all GUI apps remotely
-C	Compression	Reduces network usage; can help GUI over slow links

5.2.2. MMG cluster and recommendations for project management

As soon as collaboration is involved, follow these project tips:

- Initialise `git` and GitHub at the real start of the project.
- Keep work under `/mnt/<cluster-name>/projects/` only (nothing long-lived in your home directory except quick tests).

5.2. *Working with a remote server*

- When creating the project, set group and user permissions carefully, following the guidelines in `Server_Management.pdf`, section **Creating a New Project**.
- Maintain a `README.md` at the root of the projects tree with one short line per directory summarising the study's overarching objective.

5. AMU tools and remote servers

Listing 5.6 ssh config

```
## Alias for the host connection: either "ifb-core" or "ifb"
Host ifb-core ifb

    ## The real hostname (or IP) of the remote server
    HostName core.cluster.france-bioinformatique.fr

    ## Username used on the remote machine
    User <your-ifb-username>

    ## Path to your private SSH key
    IdentityFile ~/.ssh/id_ed25519

    ## Force SSH to use ONLY the key above
    IdentitiesOnly yes

    ## Authentication order: key, then keyboard-interactive, then password
    PreferredAuthentications publickey,keyboard-interactive,password

    ## Add this key to ssh-agent when used
    AddKeysToAgent yes

# Host configuration for the MMG cluster
Host mmg-sb-05 new-mmg-cluster
    HostName <mmg-server-hostname-or-ip>
    Port <non-default-ssh-port> # prefer a non-22 port when the cluster provides one
    User <your-mmg-username>
    IdentityFile ~/.ssh/id_ed25519
    IdentitiesOnly yes
    PubkeyAuthentication yes
    PreferredAuthentications publickey,keyboard-interactive,password
    AddKeysToAgent yes
    ForwardX11 yes
    ForwardX11Trusted yes
```

A. OneDrive on Linux

A.1. Open-source installation for Ubuntu

1. Install dependencies:

Listing A.1 onedrive deps

```
sudo apt install build-essential libcurl4-openssl-dev pkg-config git -y
```

2. Add the openSUSE Build Service repository release key:

Listing A.2 onedrive repo

```
wget -q0 - https://download.opensuse.org/repositories/home:/npreining:/debian-ubuntu-onedrive/xUbuntu_
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/obs-onedrive.gpg] https://
## update cache
sudo apt-get update
```

3. Install OneDrive:

Listing A.3 onedrive install

```
sudo apt install --no-install-recommends --no-install-suggests onedrive
```

4. Sync OneDrive:

A. OneDrive on Linux

- i. Exclude large folders and directories that are complex or not recommended for syncing: `nano ~/.config/onedrive/config`. See the contents of a working config under `onedrive-custom-config`.¹
- ii. Authenticate, then run OneDrive once:

Listing A.4 onedrive sync

```
onedrive --sync --dry-run ## optional: list folders that will be synchronised
onedrive --sync --verbose ## first initialisation; usually time-consuming; one-shot
```

A.2. Permanent monitoring: pick exactly one path

Warning 1

Do not run systemd monitoring and a OneDrive GUI / tray client at the same time. Concurrent clients fight over the same local tree and produce “already running” errors, stalled uploads, or silent conflicts. Choose **either** Listing A.5 **or** Tip 2 — never both (Figure A.1).

A.2.1. Option A — systemd + journalctl

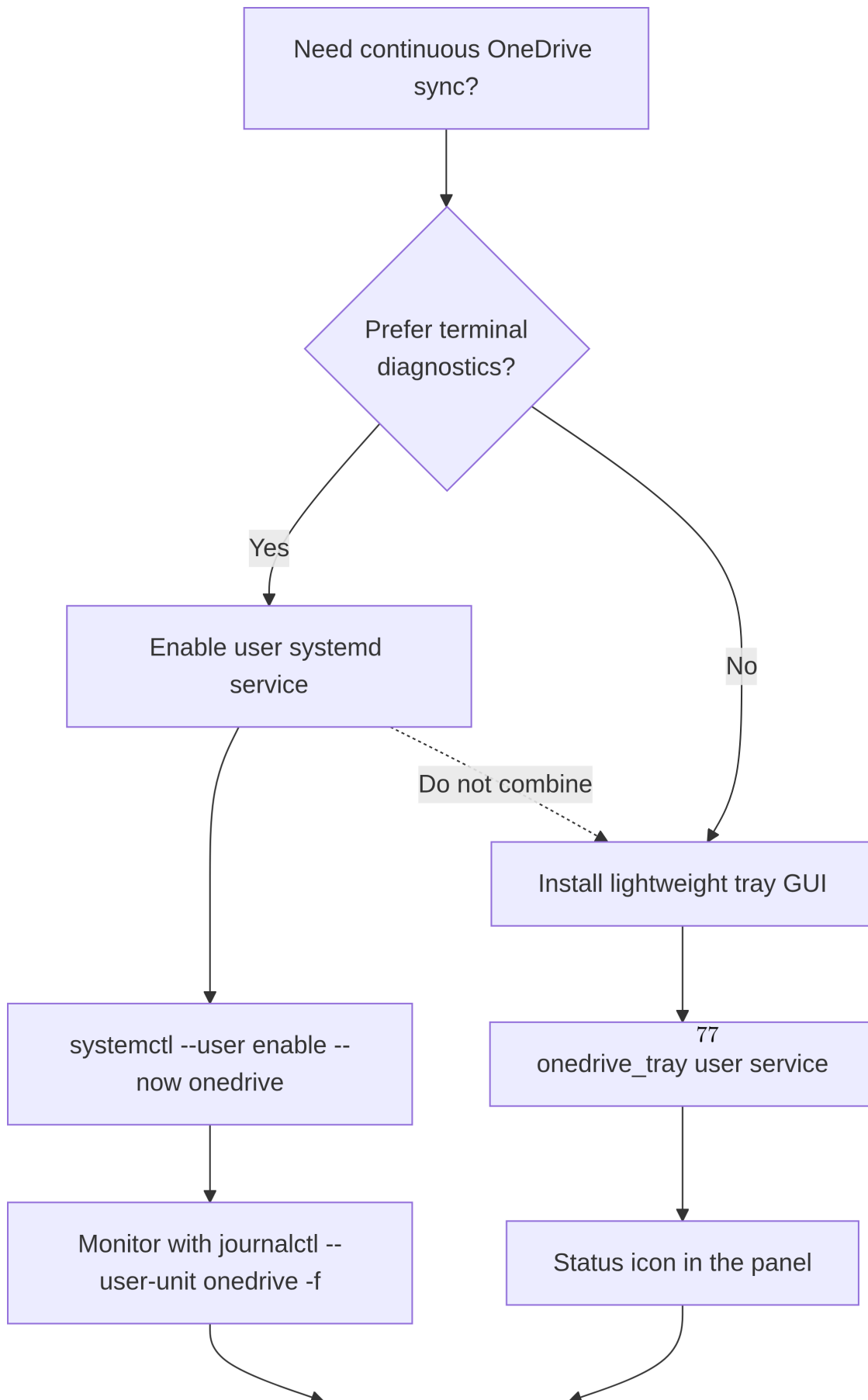
Listing A.5 onedrive systemd

```
systemctl --user enable onedrive
systemctl --user start onedrive ## backend equivalent: onedrive --monitor
```

Monitor uploads:

¹Copy the sample into `~/.config/onedrive/config`, then adjust `sync_dir` if your folder name differs.

A.2. Permanent monitoring: pick exactly one path



A. OneDrive on Linux

Listing A.6 onedrive monitor

```
journalctl --user-unit onedrive -f ## continuous; or systemctl --user status onedrive
onedrive --display-config
onedrive --version
```

A.2.2. Option B — tray GUI

Tip 2: GUIs for OneDrive monitoring

OneDrive system tray program adds a simple icon that reports OneDrive synchronisation status.

If you use this path, **disable** the systemd user service first (`systemctl --user disable --now onedrive`) so only one client runs (see Warning 1).

Listing A.7 onedrive tray

```
## install Qt 5 qmake
sudo apt install qt5-qmake qtbase5-dev

## build the executable
git clone https://github.com/DanielBorgesOliveira/onedrive_tray.git
cd onedrive_tray
mkdir build
cd build
/usr/lib/qt5/bin/qmake ../systray.pro ## prepare the Makefile
make ## build

## install so it runs at login
sudo make install ## deploy; sudo is required here
systemctl enable --user onedrive_tray.service
```

 Graphical interfaces for OneDrive

Avoid full graphical clients such as OneDriveGUI: in practice they often clash with a local `onedrive` install.

Prefer the terminal (Option A) or the lightweight tray icon (Option B in Tip 2) for clearer control over configuration.

A.3. Troubleshoot OneDrive

- Call `systemctl --user restart onedrive` after modifying the **config file** so changes take effect (Option A). To re-apply configuration retrospectively, use `onedrive --resync --sync --verbose` sparingly. **Synchronisation stops without warning when the**

A. OneDrive on Linux

config file changes; you must restart the service.

- The warning “*onedrive application is already running, please check process list*” means several concurrent `onedrive` processes are running — almost always because `systemd` and a GUI were started together (Warning 1):
 - List them: `ps aux | grep onedrive`
 - Stop the service and stray processes:

Listing A.8 Stop concurrent OneDrive clients

```
systemctl --user stop onedrive.service
pkill -f "onedrive --sync" || true
pkill -f "onedrive --monitor" || true
pgrep -af onedrive ## should return nothing before continuing
```

A.3.1. Safe recovery after a messy sync

When local and cloud trees have diverged badly, treat a restored local folder as the temporary source of truth, push it one-way, then return to two-way sync. **Do not edit files on the web or locally while a sync or resync is running.**

1. Stop every client (Listing A.8).
2. For a migration / upload-only phase, set in `~/.config/onedrive/config` (adjust `sync_dir` to your restored tree):
3. Resynchronise once (example background run):
4. Switch back to normal two-way sync: move the restored tree to your usual path if needed (`mv ~/OneDrive-restored ~/OneDrive`), restore the sample settings from `onedrive-custom-config` (`upload_only = "false"`), then:

Listing A.9 Temporary upload-only OneDrive config

```
skip_dir = "venv|\\.venv|__pycache__|node_modules|\\.next|renv|.cache"
skip_file = "\\..Rhistory$|\\.RData$|~*|.~*|*.tmp|*.swp|*.partial"
check_nosync = "true"
local_first = "true"
display_manager_integration = "true"
use_recycle_bin = "true"
force_session_upload = "true"
delay_inotify_processing = "true"
sync_dir = "~/OneDrive-restored"
upload_only = "true"
download_only = "false"
```

Listing A.10 One-way upload resync

```
nohup onedrive --sync --resync --resync-auth --verbose \
> "$HOME/onedrive-migration-resync.log" 2>&1 &
```

5. Re-enable **exactly one** monitoring path again (Warning 1): either Listing A.5 or Tip 2.

i OneDrive, Git and EOL encoding across OS

Git and OneDrive do not combine well when:

- **End-of-line encodings differ:** Linux and macOS use LF (line feed: `\n`); Windows uses CRLF (carriage return + line feed: `\r\n`).
- Linux tracks executable permissions; Windows does not (mainly an issue for shell scripts).
- `.git` directories are complex repositories and are often a poor fit for cloud sync.

A. OneDrive on Linux

Listing A.11 Final two-way resync

```
onedrive --sync --resync --verbose
```

To avoid these issues:

- i. Configure `.gitconfig` globally so both EOL styles are handled. Use `git config --global core.autocrlf input` on Linux and macOS when collaborating with Windows machines², and `git config --global core.autocrlf true` on Windows³.
- ii. For extra robustness, add a per-project `.gitattributes` file⁴:

Listing A.12 onedrive gitattributes

```
## three columns: extension, type (text or binary), EOL for text files
*.sh    text eol=lf
*.py    text eol=lf
*.js    text eol=lf
*.ts    text eol=lf
*.java  text eol=lf
```

Then apply the change with Git (this renormalises tracked files; rules in `.gitattributes` override the global `core.autocrlf` option):

Listing A.13 onedrive renormalize

```
git add --renormalize .
git status ## files affected by the EOL change
git commit -m "Normalize line endings"
```

iii. Set the editor default end-of-line encoding:

- *VS Code / Cursor*: Shift-Ctrl-P, then “Open User Settings”, search for `Files: Eol`, and set end of line to `'\n'`.

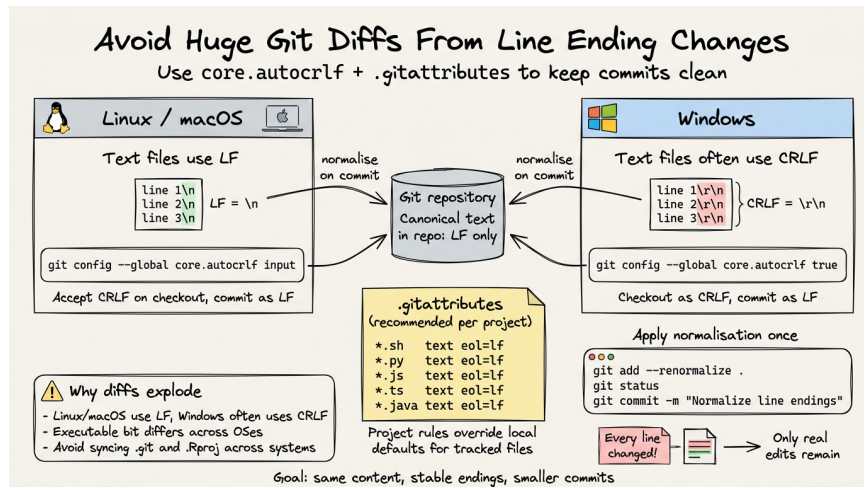


Figure A.2.: Best practices for end-of-line homogenisation across

⚠ Compatibility with editors using atomic saves

Many editors — including LibreOffice, and often VS Code / Cursor when saving project files — use **atomic save strategies** to preserve

⁴Accept CRLF as input on checkout, but always commit as LF.

⁴Constrain commits to be encoded and pushed as LF.

⁴Detailed guidance is available here.

A. OneDrive on Linux

data integrity: the editor writes a temporary file, then renames it over the original.

Listing A.14 onedrive atomic flow

```
file.qmd
↓ edit
file.qmd.tmp ## if OneDrive syncs mid-save, it may treat the file as discarded
↓ rename
file.qmd

## Architecture that causes the clash
Editor (LibreOffice, Cursor, ...)
  |
  | save (atomic)
  ▼
Local file
  |
  | filesystem event (inotify)
  ▼
onedrive client
  |
  | sync
  ▼
OneDrive cloud
  |
  | sync back
  ▼
local overwrite
```

That pattern can interfere with the onedrive client's **inotify**-based

A.3. Troubleshoot OneDrive

detection, producing editor warnings or briefly emptying file contents. Three complementary approaches:

1. **Keep active Git projects outside OneDrive.** Use OneDrive for large document stores; keep source trees under Git elsewhere (or in a path excluded from sync).
2. Pause sync while editing intensively — and remember to restart synchronisation afterwards.
3. Adjust the config to tolerate atomic saves (already in onedrive-custom-config):

Listing A.15 onedrive atomic config

```
force_session_upload = "true" ## reduce overwrites by older cloud versions mid-save
delay_inotify_processing = "true" ## wait for tmp → rename to finish before syncing
```

B. R programming guidelines

This guide consolidates package development practice using modern recommendations from the tidyverse ecosystem, especially *R Packages* (2nd edition) by Hadley Wickham and Jenny Bryan.

B.1. Package Structure

A standard R package typically contains:

- `R/`: function source files (one primary function family per file).
- `man/`: generated Rd help files (do not edit manually).
- `DESCRIPTION`: package metadata and dependency declarations.
- `NAMESPACE`: exports and imports, usually generated by `roxygen2`.
- `tests/testthat/`: automated tests.
- `vignettes/`: long-form user guides.
- `data/`: user-facing datasets (`.rda`), loaded lazily when needed.
- `data-raw/`: scripts that build derived datasets (not shipped to users directly).
- `inst/extdata/`: raw or auxiliary files users may access via `system.file()`.
- `src/`: compiled code (for example C/C++), with `LinkingTo` when relevant.
- `.Rbuildignore`: files excluded from package build.
- `.gitignore`: files ignored by Git.
- `NEWS.md`: notable user-facing changes.

B. R programming guidelines

- README.Rmd and built README.md: project overview and installation instructions.

B.2. Project Setup (End-to-End)

1. Create the package project:

```
usethis::create_package("~/path/to/pkgname")
```

2. Validate naming conventions (letters, numbers, and . only; start with a letter).

3. Activate common infrastructure:

```
usethis::use_git()
usethis::use_github()
usethis::use_mit_license("Your Name")
usethis::use_readme_rmd()
usethis::use_news_md()
```

4. Add continuous checks early:

```
usethis::use_testthat(3)
usethis::use_github_action_check_standard()
```

5. During development, use fast feedback loops:

```
devtools::load_all()
devtools::test()
devtools::document()
devtools::check()
```

B.3. Writing Functions

- Put package code in R/; avoid `source()` inside a package.

B.4. Dependencies and Namespaces

- Do not call `library()` or `require()` in package code. Prefer `pkg::fun()` at call sites, or selective `@importFrom` when a symbol is used repeatedly. Avoid broad `@import pkg` directives.
- Prefer `<-` for assignment; `reserve =` for named arguments in calls and list construction.
- Keep side effects local; for temporary global option/state changes, use `withr`.
- Use `snake_case` for functions, variables, and objects (lowercase letters and underscores). Prefer **verbs** for function names.
- File names: lowercase letters, digits, `-` and `_` only; typically one primary function family per file (for example `fct_*.R`, `fct_visu_*.R`).
- Separate **visualisation** helpers from **data-processing** functions (different files / families) so APIs stay clear.
- Format with **Air** (and respect `{lintr}` / pre-commit when configured); do not fight the formatter.
- Indent consistently (editor “reindent selection” is fine).

B.3.1. Input checking

Validate arguments early and fail with clear messages:

- Restricted string choices: prefer `rlang::arg_match()` (or base `match.arg()`) over silent defaults.
- Simple invariants: `stopifnot()` is acceptable for short checks.
- Richer type / length / range checks in packages: prefer `checkmate` (or similar helpers summarised in *Checking the inputs of your R functions*).

B.4. Dependencies and Namespaces

Use `DESCRIPTION` fields correctly:

B. R programming guidelines

- **Imports:** packages required at runtime.
- **Suggests:** optional packages used in tests, examples, or vignettes.
- **LinkingTo:** headers needed for compiled code interfaces.
- **SystemRequirements:** non-R system dependencies.

Recommended workflow:

```
usethis::use_package("dplyr", type = "Imports")
usethis::use_package("testthat", type = "Suggests")
```

In roxygen comments:

- `@export` exposes functions to users.
- `@importFrom pkg fun` imports specific functions where justified.

Prefer selective imports over broad `@import` directives.

B.5. Documentation with roxygen2

Document functions directly above their definitions and regenerate docs with:

```
devtools::document()
```

Core tags:

- `@param:` input arguments.
- `@return:` output object.
- `@examples:` runnable examples (skip expensive calls with `\dontrun{}` only when necessary).
- `@seealso:` links to related functions.
- `@family:` groups related functions.

B.6. Testing with testthat

- `@rdname` or `@describeIn`: combine multiple functions on one help page when appropriate.
- `@noRd`: still document **internal** helpers for maintainers, but keep them out of the user-facing help index.

Regenerate Rd files and `NAMESPACE` with `devtools::document()` after every meaningful documentation change — do not hand-edit `man/` in normal workflow.

For package-level documentation, create `R/<pkg>-package.R` with:

- `@keywords` `internal`
- `@name` `<pkg>-package`
- a trailing `NULL`

B.6. Testing with testthat

Initialise and run tests:

```
usethis::use_testthat(3)
devtools::test()
```

Guidelines:

- Store tests in `tests/testthat/`.
- Name files as `test-*.R`.
- Group related expectations in `test_that()` blocks.
- Use the most specific matcher available (`expect_identical()`, `expect_equal()`, `expect_match()`, etc.).
- Use snapshot/reference testing where outputs are large or complex.
- Skip selectively (`skip_if_not_installed()`, `skip_on_cran()`) rather than disabling broad test suites.

B.7. Data in Packages

Use the right location for each data type:

- `data/`: user-facing, documented datasets.
- `R/sysdata.rda`: internal package data not exposed to users.
- `inst/extdata/`: external files users can access via `system.file()`.
- `data-raw/`: scripts to build reproducible datasets.

Helpful commands:

```
usethis::use_data_raw()
usethis::use_data(my_dataset, overwrite = TRUE)
usethis::use_data(my_internal_object, internal = TRUE, overwrite = TRUE)
tools::checkRdaFiles()
```

Document datasets with roxygen `@format` and `@source`.

B.8. Build, Check, and Release

Before release:

- Run `devtools::check()` locally and in CI.
- Confirm no unintended notes/warnings/errors.
- Keep `NAMESPACE` generated (do not hand-edit unless absolutely necessary).
- Verify encoding and avoid non-ASCII issues for CRAN where possible.
- Increment versions with semantic intent:

```
usethis::use_version("patch")
```

B.9. Documenting Multiple Functions on One Help Page

- Maintain a clear `NEWS.md`.

For CRAN submission, follow current CRAN policies and perform final checks with `devtools::check()` and `devtools::check(cran = TRUE)`.

B.9. Documenting Multiple Functions on One Help Page

Default recommendation: one function family per file and one principal help topic per user concept.

Use:

- `@seealso` and `@family` for cross-links across related pages.
- `@rdname` when several functions should share one help page.
- `@describeIn` for closely related variants/methods with a shared conceptual contract.

Example:

```
#' Basic arithmetic
#'\n
#' @param x,y Numeric vectors.
#' @name arith
NULL

#' @rdname arith
add <- function(x, y) x + y

#' @rdname arith
times <- function(x, y) x * y
```

B.10. Parallel Computing in R (Practical Notes)

Check available cores:

```
parallel::detectCores()
```

Local multicore workflows:

- `parallel::mclapply()` (Unix-like systems).
- `foreach` with `%dopar%` plus `doParallel`.
- Use reproducible RNG streams for parallel simulations (for example `RNGkind("L'Ecuyer-CMRG")` with a fixed seed).

Cluster workflows (PSOCK clusters), local or remote:

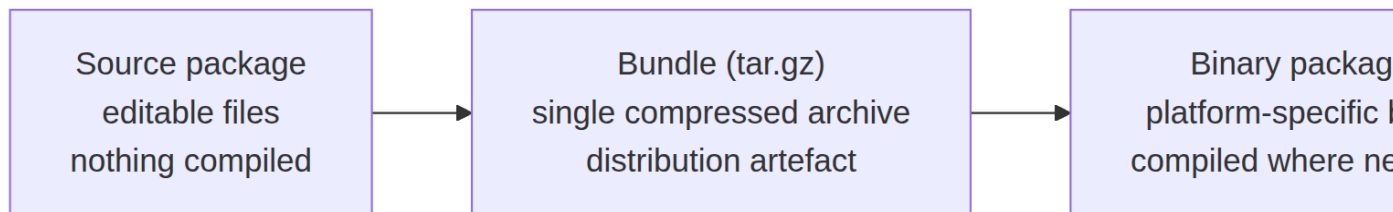
1. Create cluster with `parallel::makeCluster()`.
2. Export required objects via `parallel::clusterExport()` when needed.
3. Evaluate with `parallel::parLapply()` or related functions.
4. Stop cleanly using `parallel::stopCluster()`.

Always close clusters and backend registrations to avoid orphan workers.

B.11. Lifecycle R development

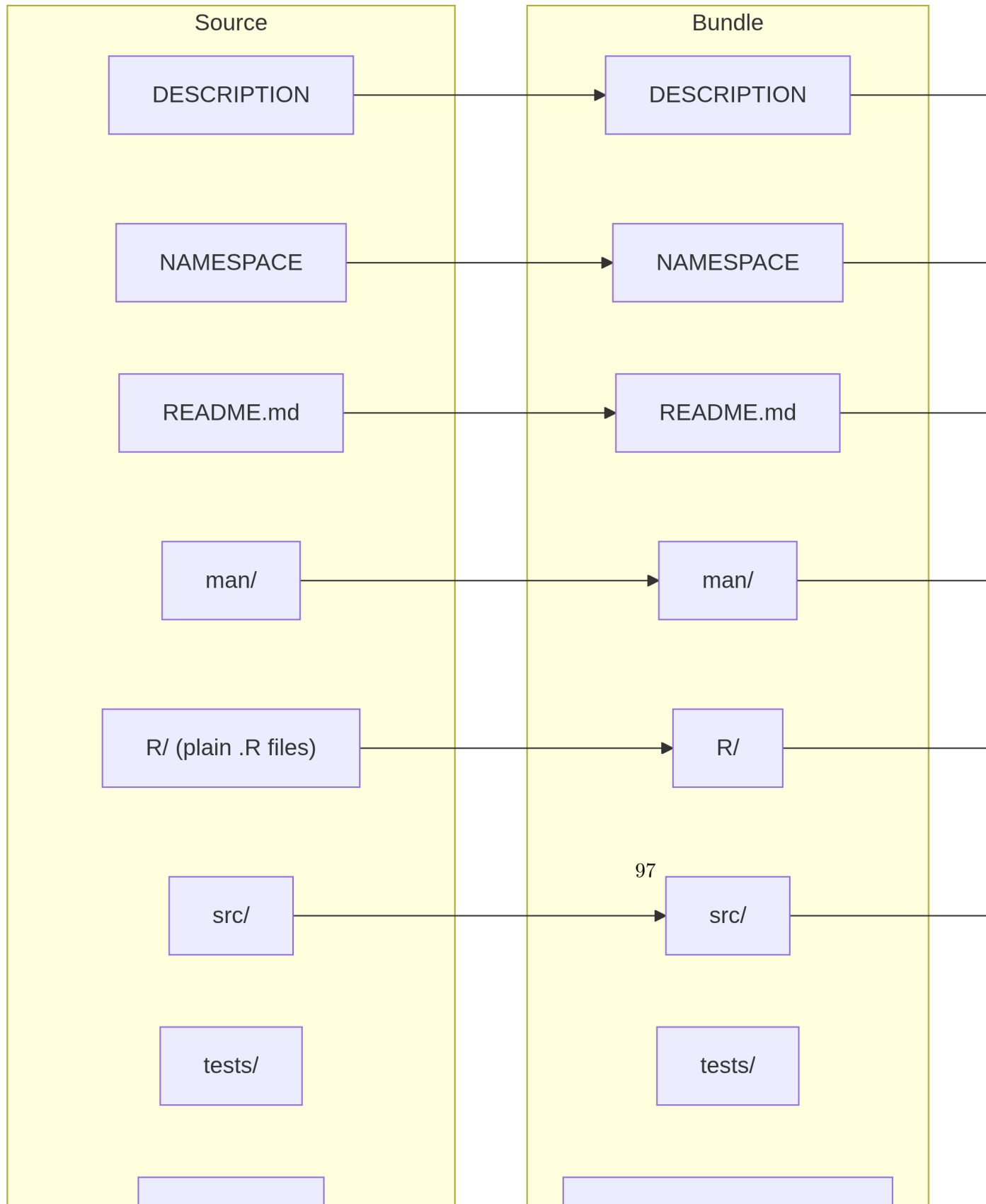
The following diagrams summarise package construction, build, installation, and loading in an Excalidraw-like flow.

B.11.1. 1) High-level package lifecycle



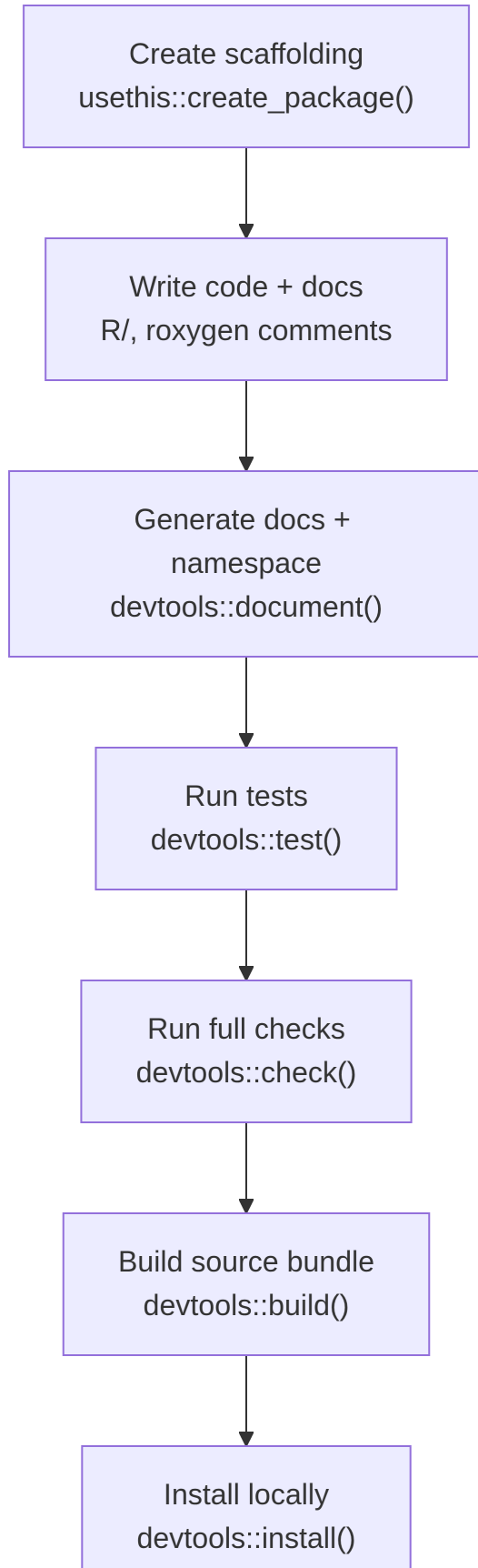
B. R programming guidelines

B.11.2. 2) What changes between source, bundle, and binary



B. R programming guidelines

B.11.3. 3) Command-driven workflow (construction to load)



C. Modern LaTeX guidelines

Modern LaTeX (and Quarto + LaTeX) practice emphasises **semantic markup**, modular projects, and packages that stay maintainable (`amsmath`, `biblatex`, `cleveref`, `siunitx`, ...).

Useful references:

- LaTeX advice by Diomidis Spinellis
- TeXtidote / style checking
- Quarto documentation
- Pandoc Markdown

For **Nature Methods**-style manuscripts in this lab, prefer TikZ / vector figures for model DAGs and equation panels; keep Mermaid for README / vignettes unless you export a journal-ready figure. See also the scientific-writing rules in `.cursorrules`.

C.1. Document structure

For theses and large papers use `\frontmatter`, `\mainmatter`, and `\backmatter`, with a master file and modular folders:

```
project/  
├─ main.tex  
├─ preamble/  
├─ chapters/
```

C. Modern LaTeX guidelines

```
|— figures/  
|— bibliography/  
└— appendices/
```

- `\include{...}` for chapters; `\input{...}` for preamble snippets.
- `\includeonly{...}` while debugging to compile a subset of chapters.

C.2. Spacing and paragraphs

Prefer semantic spacing (`\medskip`, `\bigskip`) or a global `\setlength{\parskip}{1ex}` over scattered `\.`. Set `\setlength{\parindent}{1em}` (or local `\noindent`) rather than absolute hacks when relative lengths suffice.

C.3. Mathematical typesetting

Avoid `$$ ... $$` in pure LaTeX documents. Prefer `amsmath` environments: `equation`, `align`, `split`, `multline`, `gather`, `cases`. Do **not** use obsolete `eqnarray`, or `array` merely to fake multiline displays.

Operators: never fake them with `\text{var}(X)`. Use `\operatorname{var}(X)` or declare them globally:

```
\DeclareMathOperator{\cov}{cov}  
\DeclareMathOperator{\cor}{cor}  
\DeclareMathOperator{\varop}{var}
```

Multi-character identifiers: `\mathit{Delta}` / `\mathrm{Delta}`. Descriptive subscripts upright: `x_{\mathrm{max}}`. Split long lines with `align` and indentation; stack indices with `\substack`. Upright Greek where appropriate via `upgreek`.

C.4. Text formatting

Deprecated	Preferred
<code>\bf</code>	<code>\textbf{}</code>
<code>\it</code>	<code>\textit{}</code>
<code>\em</code>	<code>\emph{}</code>

Small caps: `\textsc{sql}`. Typography: --- (em dash), -- (en dash); place footnotes after punctuation; emphasise with `\emph{}`.

C.5. Figures and tables

Prefer float placement `[tbp]` over aggressive `[h!]` / `[H]`. Inside floats use `\centering`, not a nested `center` environment. Captions: `caption` with `singlelinecheck=off` when lists appear in captions.

Tables: `booktabs`, `longtable`, `tabularx`. See also Generating better LaTeX tables and LaTeX Tables generator. In Quarto / HTML workflows, `kableExtra` / paged tables are often enough.

C.6. Bibliography and citations

Prefer `biblatex` + `biber` over legacy BibTeX-only workflows:

```
\usepackage[
  backend=biber,
  style=authoryear
]{biblatex}
```

C. Modern LaTeX guidelines

Use `csquotes` and `\enquote{...}`. Page-aware cites: `\autocite[p.~8]{LR89}`. Validate / clean `.bib` files (remove noisy fields such as `abstract`, `file`, `urldate` when unused) with tools like `biblatex-linter`, BibTeX Tidy, and sources such as DBLP.

C.7. Cross-references

```
\usepackage{cleveref}
```

Prefer `\cref{eq:model}` / `\Cref{...}` / `\crefrange{...}` over manual `Eq.~(\ref{...})`.

C.8. Lists

```
\usepackage{enumitem}
\begin{enumerate}[label=(\alph*)]
```

C.9. Recommended packages

Domain	Packages
Mathematics	<code>amsmath</code> , <code>amssymb</code> , <code>mathtools</code> (+ <code>systeme</code> / domain packs if needed)
Theorems	<code>amsthm</code> or <code>ntheorem</code> ; boxes via <code>tcolorbox</code> / <code>mdframed</code>
Units	<code>siunitx</code> (<code>\num{1e6}</code> , <code>\unit{\micro\meter}</code> , ...)

Domain	Packages
Graphics	<code>tikz</code> , <code>pgfplots</code> (and <code>TikZ</code> for Nature Methods DAGs)
Code	<code>listings</code> or <code>minted</code> (needs <code>Pygments</code>)

C.10. Quarto LaTeX mapping

LaTeX	Quarto / Markdown
<code>\citep{Smith2020}</code>	<code>[@Smith2020]</code>
<code>\citet{Smith2020}</code>	<code>@Smith2020</code>
<code>\(x + y \)</code>	<code>\$x + y\$</code>
numbered display math	<code>\$\$... \$\$ {#eq-square} then @eq-square</code>
<code>\section{Introduction}</code>	<code># Introduction</code>
<code>\href{URL}{Text}</code>	<code>[Text](URL)</code>
<code>\url{URL}</code>	<code><URL></code>
<code>\textit / \textbf</code>	<code>*...* / **...**</code>
<code>\footnote{...}</code>	<code>^[...] or [^id]</code>
figure / table refs	<code>@fig-... / @tbl-...</code>

Example Quarto table chunk (illustrative only):

```

::: {#tbl-summary .cell tbl-cap='Summary statistics'}
:::

```

Then: See `@tbl-summary`.

C.11. Compilation and quality

- Automate with `latexmk -pdf main.tex`.
- Build figures as `standalone` documents when useful.
- Lint / review: `TeXtidote`, `ChkTeX`, `refcheck`, `showkeys`.

C.12. Anti-patterns

Avoid `$$...$$` in classic LaTeX sources, `eqnarray`, decorative `\`, deprecated font switches, hardcoded visual markup, and habitual `[H]` floats. Prefer semantic structure, modular files, modern packages, and Quarto-compatible syntax when the deliverable is a website or vignette.